

RAPPORT DE STAGE DE FIN D'ETUDES

5ème Année en Ingénierie Informatique et Réseaux

Modélisation de l'attrition des clients dans le secteur bancaire marocain

Réalisé par :
Mohamed Salem BERRADA

Tuteur (s) :

Encadrant Professionnel : Mme Amina MOHANDIS
Encadrant Pédagogique : Pr. Said NOUH

Au sein de (Organisme d'accueil) :



Dédicaces

A mes chers parents :

Ce mémoire de fin d'études représente la culmination d'une période intense de travail et d'apprentissage. En votre honneur, je consacre ces pages avec une profonde gratitude.

Votre soutien indéfectible, vos précieux conseils et votre amour inconditionnel ont été des piliers essentiels tout au long de mon projet. Votre présence a été une source inestimable de force et de motivation. Grâce à vous, j'ai pu persévérer, dépasser mes limites et atteindre les objectifs que je me suis fixé.

Du fond du cœur, Je vous remercie d'avoir été les architectes de mon parcours. Ce rapport de stage de fin d'année est donc une humble dédicace en reconnaissance de votre dévouement et de votre amour inconditionnel. Vous êtes mes modèles, mes mentors, et je suis profondément honoré de vous avoir comme parents.

Avec tout mon amour et une gratitude infinie,

A mes précieux professeurs :

Ce mémoire de fin d'études symbolise mon apprentissage et ma formation académique. Je vous dédie ces pages en témoignage de ma profonde gratitude. Vos enseignements rigoureux, votre expertise et vos encouragements constants sont des éléments clés tout au long de cette expérience. Votre influence est une source inestimable d'inspiration et de motivation. Votre préparation solide et votre confiance en moi ont été les fondations sur lesquelles j'ai pu bâtir ma confiance pour affronter ce stage.

C'est un honneur pour moi de compter sur votre guidance et mentorat, et je suis profondément reconnaissant de toutes les leçons que j'ai pu apprendre de vous.

Avec mon plus grand respect,

Mohamed Salem BERRADA

Remerciements

Je tiens à exprimer mes profondes gratitude et respectueuse reconnaissance à mon encadrant pédagogique : **Pr. Said NOUH**, pour sa bonne volonté d'accepter de m'encadrer, pour tout le temps qu'il m'a octroyé et pour tous les conseils qu'il m'a prodigués.

Je remercie aussi mon encadrante professionnelle de la société **Attijariwafa Bank : Mme. Amina MOHANDIS**, pour ses directives précieuses, et pour la qualité de son suivi durant toute la période de stage.

Aussi un remerciement pour les membres du jury pour l'honneur qu'ils m'ont accordé en acceptant d'évaluer mon travail.

Sans oublier de remercier les professionnels rencontrés au sein d'AWB ; **Mohamed GHALI, Youssra SIRAJ EL HAQ, Youssef OUMOUHOU** et **Ilyas RYAH** pour leur patience, leur disponibilité et surtout leurs judicieux conseils, qui ont contribué à alimenter ma réflexion.

Enfin, un vif remerciement ainsi à toutes les personnes qui ont contribué, de près ou de loin, à l'élaboration de cette mémoire ainsi qu'à la réussite de cette formidable année universitaire.

Mohamed Salem BERRADA

Résumé

Avec la croissance des données clients disponibles dans le secteur bancaire, leur exploitation est devenue essentielle pour optimiser à la fois la qualité du service et la rentabilité. L'un des principaux défis des banques est l'attrition des clients, un phénomène en hausse ces dernières années. En effet, la loi marocaine sur la mobilité bancaire (circulaire n° 5/W/2019 de Bank Al-Maghrib) a pour objectif de faciliter le changement de banque pour les clients, en leur permettant de transférer leurs comptes et leur opérations bancaires d'une banque à une autre de manière plus simple et plus rapide.

Cette nouvelle loi vise à renforcer la concurrence entre les banques et à offrir aux clients un plus grand choix de services bancaires tout en accentuant le risque de perte de clientèle et un taux d'attrition plus élevé. Face à cet enjeu, ce mémoire propose d'explorer et de modéliser l'attrition des clients chez Attijariwafa bank en développant un modèle de prédiction capable d'identifier les clients susceptibles de partir. Pour cela, plusieurs algorithmes d'apprentissage supervisé seront mis en œuvre, et le plus performant sera retenu en fonction de son taux de précision.

Mots clés : exploitation des données, l'attrition des clients, modèle de prédiction, apprentissage supervisé.

Abstract

With the growth of customer data available in the banking sector, leveraging it has become essential to optimize both service quality and profitability. One of the main challenges for banks is customer churn, a phenomenon on the rise in recent years. Indeed, the Moroccan law on banking mobility (Bank Al-Maghrib circular no.5/W/2019) aims to facilitate bank switching for customers by allowing them to transfer their accounts and banking operations from one bank to another more easily and quickly.

This new law aims to strengthen competition between banks and offer customers a wider choice of banking services, while increasing the risk of customer loss and a higher churn rate. Faced with this challenge, this paper proposes to explore and model customer churn at Attijariwafa Bank by developing a predictive model capable of identifying customers likely to leave. To do this, several supervised learning algorithms will be implemented, and the most efficient one will be selected based on its precision rate.

Keywords : data mining, customer churn, predictive model, supervised learning

Liste des tableaux :

Tableau 1 : Plan déroulement du projet.....	12
Tableau 2 : Type de Machine Learning.....	18
Tableau 3 : Dataset « Profil & Produit ».....	23
Tableau 4 : Dataset « connexion digital »	24
Tableau 5 : Dataset « opération bancaire ».....	24
Tableau 6 : Dataset « Solde & CMC »	25
Tableau 7 : Dataset « Base Attrition »	25
Tableau 8 : Tableau comparatif des datasets.....	26
Tableau 9 : Synthèse comparative	33
Tableau 10 : Variables dérivées	39
Tableau 11 : Catboost Hyperparamètres.....	48
Tableau 12 : Matrice de Confusion	49
Tableau 13 : Formules DAX.....	68

Liste des figures :

Figure 1: AWB Logo	4
Figure 2 : AWB Services	5
Figure 3: AWB Organigramme.....	6
Figure 4: QlikView Interface.....	7
Figure 5: CRISP-DM Methodology	10
Figure 6 : Machine Learning.....	19
Figure 7 : Taille du dataset fusionné	26
Figure 8 : Importation des bibliothèques.....	27
Figure 9 : Script « Attrition règle 1 »	28
Figure 10 : Script Attrition règle 2.....	29
Figure 11 : Résultat du 1 er test « Random Forest »	30
Figure 12 : Script « Random Forest + SMOTE »	31
Figure 13 : Résultat « Random Forest avec SMOTE »	32
Figure 14 : Script « Random Forest + Undersampling »	32
Figure 15 : Résultat « Random Forest avec Undersampling »	33
Figure 16 : Affichage des valeurs manquantes	34
Figure 17 : Script « imputation âge (1) »	35
Figure 18 : Graphe « distribution âge (1) »	36
Figure 19 : Script « Imputation âge(2) »	37
Figure 20 : Graphe « distribution âge (2) »	37
Figure 21 : Conversion des colonnes	38
Figure 22 : Dataset Final.....	40
Figure 23 : Pycaret logo	41
Figure 24 : Evaluation des modèles	42
Figure 25 : CatBoost logo	43
Figure 26 : Principe du Boosting	43
Figure 27 : Extrait du dataset	44
Figure 28 : Script « encodage variable catégorielle »	44
Figure 29 : Séparation X et Y - Feature Importance	45
Figure 30 : Script « Feature Importance ».....	45
Figure 31 : Feature Importance.....	46
Figure 32 : Résultat du Random Forest avec 72 variables.....	46
Figure 33 : Extrait dataset (28 variables).....	47
Figure 34 : Séparation des données – CatBoost.....	47
Figure 35 : Entraînement du modèle Catboost	48
Figure 36 : Script « Catboost »	52
Figure 37 : Résultat « Catboost »	53
Figure 38 : Script « Graphe Seuil de décision ».....	54
Figure 39 : Graphe « Seuil de décision »	54
Figure 40 : Script « Seuil de décision ».....	55
Figure 41 : Résultat du Modèle Catboost avec seuil de décision	55
Figure 42 : Test & Train Accuracy.....	56
Figure 43 : Script « Courbe d'apprentissage »	56
Figure 44 : Graphe « Courbe ROC »	57
Figure 45 : Catboost « Règle 1 »	59
Figure 46 : Catboost « Règle 2 »	59

Figure 47 : Streamlit Logo.....	60
Figure 48 : Script « sélection modèle ».....	61
Figure 49 : Sélection du modèle selon la règle.....	62
Figure 50 : Script « chargement fichier »	62
Figure 51 : Chargement du fichier	62
Figure 52 : Vérification du fichier	63
Figure 53 : vérification du fichier (Echec).....	64
Figure 54 : vérification du fichier (succès)	64
Figure 55 : Script « Lancer prédiction »	65
Figure 56 : Résultats de la prédiction avec colonnes ajoutées	65
Figure 57 : Script « Aperçu Résultat »	66
Figure 58 : Taux d'attrition et nombre de clients à risque	66
Figure 59 : Script « Exportation et Visualisation Power BI »	67
Figure 60 : Boutons de téléchargement et d'ouverture Power BI.	67
Figure 61 : Vue Macro – Attrition	69
Figure 62 : Vue Individuelle - Attrition.....	70

Table des matières

Introduction Générale	1
Chapitre I : CADRAGE ET CONDUITE DU PROJET	3
1.1 Introduction	4
1.2 Organisme d'accueil :	4
1.2.1 Historique	4
1.2.2 Services	5
1.2.3 Organigramme de l'organisme d'accueil	6
1.3 Présentation du projet :	7
1.3.1 Analyse de l'existant.....	7
1.3.2 Problématique	8
1.3.3 Objectifs.....	9
1.3.4 Contraintes.....	9
1.4 Spécification des besoins.....	9
1.5 Méthodologie de travail	10
1.6 Planification Prévisionnelle	12
1.7 Conclusion	12
Chapitre II : FONDAMENTAUX DE L'APPRENTISSAGE AUTOMATIQUE.....	13
2.1 Introduction.....	14
2.2 Concepts généraux du ML :	14
2.2.1 Apprentissage automatique	14
2.2.2 Types d'apprentissage en Machine Learning.....	14
2.2.3 Objectifs des méthodes d'apprentissage	15
2.3 Applications du Machine Learning dans la gestion de l'attrition :	18
2.3.1 Pourquoi le ML pour prédire l'attrition ?	19
2.3.2 Exemples concrets d'utilisation du ML pour l'attrition	20
2.3.3 Les différents types de données utilisées pour la prédiction de l'attrition	21
2.4 Conclusion	21
Chapitre III : REALISATION DE LA SOLUTION	22
3.1 Introduction	23
3.2 Sources et nature des données :	23
3.2.1 Types de données exploitées	23
3.2.2 Format des données	26

3.3 Préparation des données :	26
3.3.1 Environnement technique et bibliothèques Python utilisées	27
3.3.2 Labellisation de la variable cible (Attrition)	28
3.3.3 Test initial sans Feature Engineering	30
3.3.4 Nettoyage et filtrage	34
3.3.5 Gestion des valeurs manquantes	34
3.3.6 Transformation des variables	38
3.3.7 Construction des variables dérivées (Feature Engineering)	38
3.3.8 Finalisation du dataset	39
3.4 Modélisation :	41
3.4.1 Test des modèles	41
3.4.2 Choix de l'algorithme : Catboost	42
3.4.3 Réduction du nombre de variables par importance	44
3.4.4 Entraînement du modèle Catboost	47
3.5 Évaluation du modèle :	49
3.5.1 Définitions des principales métriques	49
3.5.2 Résultats du modèle CatBoost	52
3.5.3 Interprétation des résultats	53
3.5.4 Evaluation des performances et optimisation	54
3.5.5 Comparaison des deux règles de labellisation	58
3.6 Déploiement de la solution :	60
3.6.1 Objectifs du déploiement	61
3.6.2 Fonctionnement détaillé de l'application Streamlit	61
3.6.3 Visualisation des résultats avec Power BI	68
3.6.4 Avantages de la solution	70
3.7 Résultats et bénéfices attendus	71
3.8 Limites de la solution proposée	72
3.9 Conclusion	73
Conclusion Générale	74

Introduction Générale

À l'ère de la digitalisation, de l'intelligence artificielle et des avancées technologiques, la concurrence dans le secteur des services au Maroc n'a jamais été aussi intense. Cette dynamique pousse les entreprises marocaines à repenser leur relation client pour se démarquer.

Afin de gagner un avantage concurrentiel, il est essentiel pour les entreprises de mieux comprendre le profil de leurs clients pour leur proposer des offres sur mesure et anticiper leurs besoins. Par ailleurs, la culture d'entreprise évolue rapidement, obligeant les organisations à revoir leurs stratégies pour s'adapter aux nouvelles attentes des consommateurs. Auparavant axées sur une logique de production et de vente de masse, les entreprises marocaines se concentrent désormais sur la création de valeur client à travers une approche centrée sur l'expérience et la fidélisation à long terme.

Le marketing relationnel et la fidélisation des clients jouent donc un rôle clé dans la croissance des entreprises au Maroc. Face à des clients de plus en plus volatils et exigeants en matière de personnalisation, les entreprises adoptent des politiques de rétention, sachant qu'il est bien moins coûteux de conserver un client que d'en acquérir un nouveau. Des études, comme celle de Frederick Reichheld (1996), montrent que le coût d'acquisition d'un nouveau client peut être jusqu'à cinq fois supérieur à celui de sa fidélisation.

Le secteur bancaire au Maroc, tout comme celui des télécoms il y a quelques années, connaît une transformation majeure. Avec l'émergence de nouveaux acteurs tels que les banques en ligne, les fintechs et les néobanques, la concurrence s'intensifie, notamment sur les services de paiement et de digital banking.

Aujourd'hui, le compte bancaire est le produit le plus susceptible d'entraîner un changement de banque. Selon Bank Al-Maghrib, plus de 70% des Marocains disposent d'un compte bancaire, mais la mobilité bancaire s'accélère avec l'adoption croissante des solutions digitales. Les clients sont désormais plus exigeants et peuvent changer d'établissement bancaire plus facilement.

Dans ce contexte, la rétention client devient une priorité stratégique pour les banques marocaines. Pour y parvenir, elles doivent analyser le comportement de leurs clients afin d'identifier ceux susceptibles de partir – un phénomène appelé **attrition client**. La prédiction de l'attrition permet d'anticiper les départs, d'adapter les offres et de renforcer la fidélité.

Pour réduire l'attrition, les banques marocaines peuvent exploiter leurs vastes bases de données grâce à des outils avancés comme le **data warehousing**, le **data mining** et les logiciels de gestion de campagnes. L'exploration de données (data mining) permet de détecter des tendances et des corrélations cachées, offrant ainsi une meilleure connaissance client.

Cependant, sans une compréhension approfondie du marché marocain et des spécificités locales, ces analyses restent inefficaces. En combinant data mining et expertise sectorielle, les banques peuvent anticiper les risques d'attrition, personnaliser leurs services et renforcer leur position concurrentielle.

Le présent projet, mené dans le cadre de mon stage de fin d'études au sein d'Attijariwafa Bank, a pour objectif de développer une solution d'analyse et de prédiction de l'attrition. L'approche adoptée repose sur trois axes majeurs : la collecte des données des clients, l'analyse des données, et la mise en œuvre de modèles prédictifs permettant d'anticiper le risque de départ des clients.

Ce mémoire est subdivisé en trois chapitres. Dans le premier chapitre, nous présentons le contexte général du projet. Ici, la problématique liée à la fidélisation, les objectifs visés et la méthodologie adoptée. Dans le second chapitre, nous introduisons les concepts clés du Machine Learning, notamment la classification supervisée et non supervisées, quelques algorithmes utilisés pour l'attrition et quelques exemples concrets d'utilisation du ML pour prédire l'attrition. Enfin dans le troisième chapitre, détaille la mise en œuvre pratique : préparation des données, labellisation heuristique, entraînement du modèle, évaluation des résultats et création d'un outil de scoring destinés aux équipes métier.



Chapitre I : CADRAGE ET CONDUITE DU PROJET

1.1 Introduction

Cette étude entre dans le cadre de la préparation d'un mémoire de fin d'études en vue d'obtention du diplôme d'ingénieur Informatiques et Réseaux au sein de l'Ecole Marocaine des Sciences de l'Ingénieur.

Afin de mettre en évidence le contexte dans lequel s'inscrit notre projet, ce chapitre présente deux points :

Dans un premier temps, nous présentons l'organisme d'accueil dans lequel notre stage a été effectué. Dans un second lieu, nous abordons le contexte et les objectifs du projet.

Enfin, nous concluons ce chapitre par la planification du projet et la spécification des besoins.

1.2 Organisme d'accueil :

1.2.1 Historique



Figure 1: AWB Logo

Attijariwafa Bank est le premier groupe bancaire et financier du Maghreb, et l'un des principaux acteurs du secteur bancaire sur le continent africain. Le groupe est issu de la fusion, en 2003, de deux grandes institutions : la Banque Commerciale du Maroc (BCM) et Wafabank. Il est aujourd'hui une filiale du groupe Al Mada, un fonds d'investissement panafricain à capitaux privés.

Le siège du groupe se situe à Casablanca, au Maroc. Attijariwafa Bank est présente dans plus de 25 pays, principalement en Afrique (Tunisie, Sénégal, Côte d'Ivoire, Cameroun, Égypte, etc.) ainsi qu'en Europe (France, Belgique, Allemagne, Espagne, Italie) à travers ses filiales [0].

► Quelques chiffres clés :

- +10 millions de clients,
- Présence dans **26 pays**, principalement en Afrique, mais aussi en Europe (France, Belgique, Allemagne),
- Près de **5 000 agences**,
- Plus de **20 000 collaborateurs**.

1.2.2 Services

Le groupe offre une large gamme de produits et services à travers plusieurs pôles :

- **Banque de détail** : comptes courants, épargne, crédits à la consommation, cartes bancaires, packages.
- **Banque privée et de gestion de patrimoine** : gestion personnalisée de portefeuille, conseil patrimonial.
- **Banque d'affaires et corporate** : financements structurés, ingénierie financière, fusions & acquisitions.
- **Services digitaux** : application mobile, banque en ligne, chatbot, outils de souscription à distance.
- **Banque participative (Sharia-compliant)** : via sa filiale Bank Assafa.
- **Assurance** : assurance-vie, assurance santé, prévoyance (via Wafa Assurance).
- **Microfinance et inclusion financière** : services via la Fondation Attijariwafa Bank.



Figure 2 : AWB Services

1.2.3 Organigramme de l'organisme d'accueil

Le service Pilotage de performance BDME, au sein de laquelle nous avons effectué notre stage de PFE, s'inscrit au sein du pôle Finances Groupe. Ce service est chargé de :

- La préparation du comité CDC : (production des Reportings, Arbres d'analyse, Fiche KPIs commerciaux ;
- La préparation des documents d'analyse post Comité CDC (Ex : Arbre PNB/ Client pour Banque privée, Drill down/ agence, . . .) ;
- Le pilotage de la performance Crédit Immobilier (utilisation de la nouvelle Base de données risque, PNB, commercial, . . .) ;
- L'étude de la performance du réseau jeune ;
- Le processus budgétaire Retail ;
- L'analyse du manque à gagner sur les commissions ;
- Les calculs du risque par marché et par région ;
- Le rapprochement des données analytiques VS données comptables ;
- La simulation des nouveaux Taux de Cession Interne ;
- Le pilotage du PNB AWBE pour MRE.



Figure 3: AWB Organigramme

1.3 Présentation du projet :

1.3.1 Analyse de l'existant

Avant de définir une démarche analytique dédiée à l'étude de l'attrition, il est essentiel d'évaluer les outils et processus actuellement utilisés au sein de la banque pour le suivi de la clientèle.

► Outils et processus existants

QlikView fonctionne sur le principe d'un moteur d'association in-memory, permettant aux utilisateurs d'explorer les données librement sans suivre un parcours de requêtes prédéfini. Il est souvent connecté à des **datamarts** construits en amont, contenant des données nettoyées et organisées autour de thématiques métiers.

Les utilisateurs (équipes marketing, risque...) accèdent donc à des datamarts via QlikView pour :

- Visualiser l'évolution de la base clients,
- Suivre les indicateurs d'attrition ou d'engagement,
- Analyser la performance des actions de rétention et des campagnes ciblées.

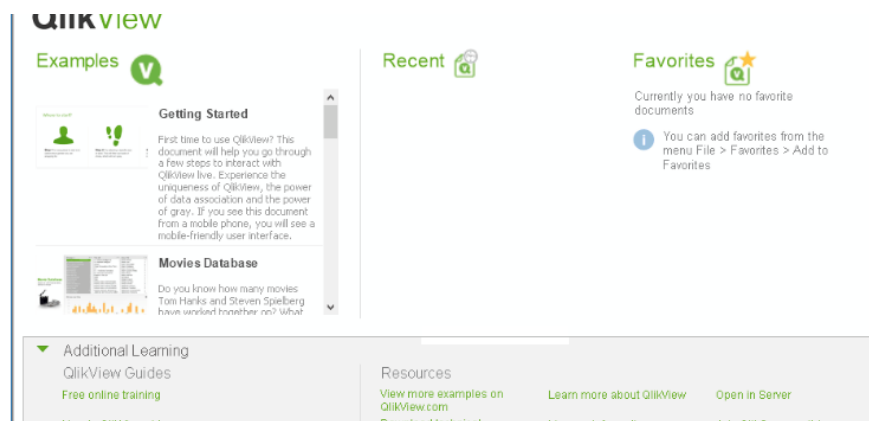


Figure 4: QlikView Interface

Attijariwafa Bank utilise la solution **QlikView** pour la visualisation et l'analyse des données. Les utilisateurs métier accèdent à des **datamarts thématiques** construits par les équipes BI (Business Intelligence), contenant des données clients, produits, transactions et interactions.

Plusieurs dashboards QlikView permettent de suivre :

- L'évolution du nombre de clients actifs/inactifs,
- Les taux de rétention par région ou segment,
- La performance des campagnes de fidélisation.

Les indicateurs d'attrition sont calculés à partir de règles métier simples (ex : inactivité pendant x mois), mais ne reposent pas sur des modèles prédictifs.

► Limitations observées

- L'analyse actuelle est **essentiellement descriptive** et fondée sur des indicateurs agrégés. Elle permet de **constater** l'attrition, mais **pas de l'anticiper**.
- **Pas de segmentation dynamique** basée sur les comportements réels (fréquence des opérations, usage digital, etc.).
- L'accès aux données reste structuré autour des datamarts existants, ce qui peut limiter les croisements de données utiles pour une approche plus exploratoire.

Cette analyse de l'existant justifie le besoin de compléter les outils actuels par une **approche data science orientée prédiction et personnalisation**, comme développée dans ce projet.

1.3.2 Problématique

Dans un contexte où le **coût d'acquisition d'un nouveau client** est de plus en plus élevé, la **rétenition** devient une priorité. Les enjeux sont :

- Préserver les revenus récurrents et la rentabilité,
- Mieux comprendre les besoins des clients pour ajuster les services,
- Identifier les signaux faibles de départ et agir proactivement,
- Créer une relation durable grâce à des offres personnalisées.

La **valorisation des données client** et la **modélisation de l'attrition** permettent de faire évoluer la banque vers une logique plus proactive, centrée sur la satisfaction et la fidélité.

Malgré les efforts en matière de transformation digitale et de services personnalisés, la banque fait face à un taux d'attrition non négligeable dans certains segments. Il devient donc crucial d'identifier en amont les clients à risque, de comprendre les raisons sous-jacentes à leur départ, et de mettre en place des actions ciblées.

Problématique posée :

Comment exploiter les données clients disponibles pour construire un modèle de classification permettant d'identifier de manière fiable les clients à risque de churn (attrition) ?

1.3.3 Objectifs

L'objectif principal de ce projet est de mettre en œuvre une démarche analytique permettant de comprendre et d'anticiper le phénomène d'attrition client au sein d'Attijariwafa Bank. Pour cela, les objectifs opérationnels sont les suivants :

1. **Prédiction de l'attrition** : développer des modèles supervisés pour prédire la probabilité qu'un client quitte la banque ;
2. **Scoring et priorisation** : attribuer à chaque client un score de risque d'attrition afin de permettre des actions marketing ciblées et efficaces ;
3. **Visualisation des résultats.**

1.3.4 Contraintes

La réalisation de ce projet doit tenir compte d'un certain nombre de contraintes détaillées ci-dessous :

- Le respect de la période fixée pour la finalisation du projet ;
- *Données manquantes* : Certaines variables clés (ex : feedback client, raison de clôture de compte) peuvent être absentes ou incomplètes.;
- *Biais de sélection* : Les données historiques peuvent ne pas inclure des clients récemment acquis ou des segments sous-représentés.
- *Données déséquilibrées* : Le taux d'attrition est souvent faible (ex : 5-10%), ce qui peut fausser les modèles prédictifs (*class imbalance problem*).
- Les données issues des études réalisées ainsi que les documents internes de la société sont à titre confidentiel, leur utilisation doit être limitée aux finalités du projet ;

1.4 Spécification des besoins

Besoins fonctionnels :

Les besoins fonctionnels définissent les actions que le système ou l'analyse doit permettre d'accomplir dans le cadre de la gestion de l'attrition :

- Collecte et Préparation des Données ;
- **Modélisation prédictive** du risque d'attrition via des algorithmes de machine learning ;
- **Génération d'un score individuel d'attrition** pour chaque client ;
- **Visualisation des résultats** via des dashboards ou rapports interactifs (type Power BI / Tableau).

Besoins non-fonctionnels :

Ces besoins définissent les contraintes techniques, de performance et d'intégration du projet :

- **Performance** : temps de traitement raisonnable même sur un grand volume de clients.
- **Interprétabilité** : les résultats des modèles doivent être compréhensibles par les métiers (notamment pour la justification des décisions marketing).
- **Sécurité et confidentialité** : respect de la politique RGPD / DCP (Données à Caractère Personnel).
- **Modularité** : architecture souple permettant des ajustements futurs (nouveaux modèles, nouvelles sources de données...).

1.5 Méthodologie de travail

Le projet a été conduit selon une approche itérative et agile, inspirée du cycle CRISP-DM (Cross Industry Standard Process for Data Mining), particulièrement adapté aux projets de data science.

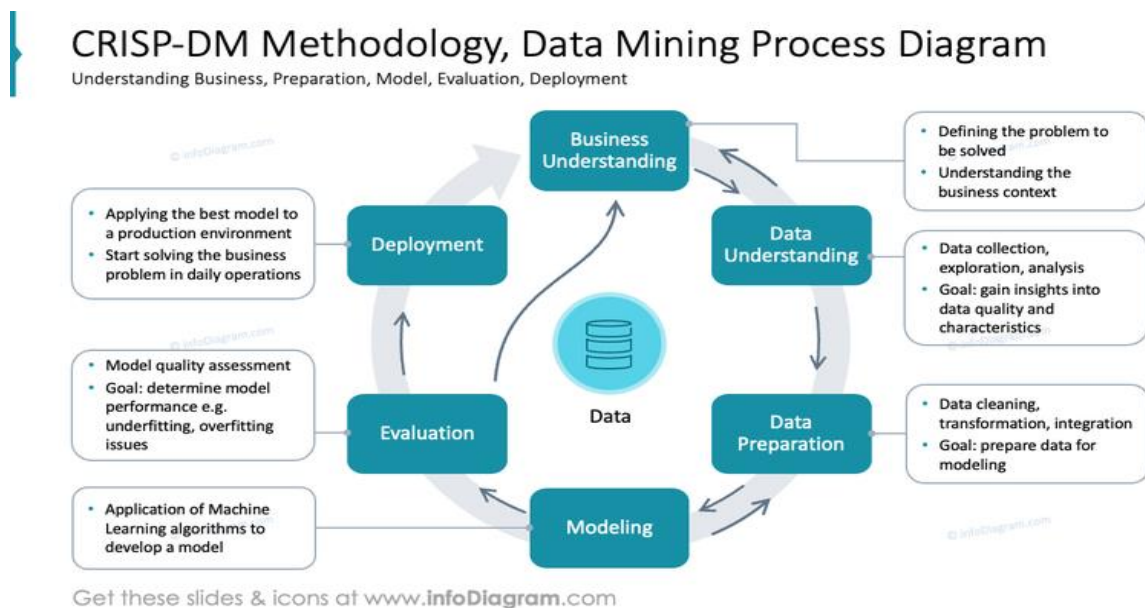


Figure 5: CRISP-DM Methodology

Cette méthodologie a été choisie pour plusieurs raisons :

- **Adaptée aux projets analytiques et de data science** : Contrairement aux méthodes classiques de développement logiciel (en V ou cycle en cascade), **CRISP-DM** est spécifiquement conçue pour les projets exploratoires basés sur les données, ce qui correspond parfaitement au sujet de l'attrition client ;
- **Approche itérative et flexible** : Le processus **n'est pas linéaire**, il permet de revenir à des étapes précédentes (ex : revenir à la préparation des données si un modèle performe mal). Cela a été très utile pour ajuster les variables utilisées dans le clustering ou les modèles prédictifs ;
- **Structuration claire en 6 étapes** : La structuration en étapes claires (compréhension métier, données, préparation, modélisation, évaluation, déploiement) offre un cadre de travail rigoureux tout en laissant une certaine souplesse ;
- **Réutilisable et standardisé** : C'est une méthodologie **reconnue dans l'industrie** et réutilisable pour d'autres cas d'usage : churn, scoring de crédit, segmentation marketing... Elle favorise également une bonne communication entre data scientists, managers et équipes métier.

1.6 Planification Prévisionnelle

La clé principale de la réussite d'un projet est un bon planning. En effet, le planning aide à bien subdiviser le travail et séparer les tâches à réaliser, il offre une meilleure estimation et gestion de temps nécessaire pour chaque tâche. De plus, il donne assez de visibilité permettant d'estimer approximativement la date d'achèvement de chaque tâche.

Pour un stage d'une durée de 6 mois, nous avons estimé de réaliser notre projet dans cette durée approximativement. Le tableau ci-dessous montre le planning que nous avons adapté pour mener à bien notre réalisation des différentes parties du projet :

Période	Tâches prévues
Mars	- Prise de connaissance du sujet, de l'environnement métier et technique - Collecte des besoins auprès des équipes métier - Exploration initiale des données (crédits, dépôts, services, digital, etc.)
Avril	- Prétraitement des données (nettoyage, gestion des valeurs manquantes, normalisation) - Labellisé le dataset final avec les règles métiers (Scoring Heuristic)
Mai	- Début des travaux sur la modélisation de l'attrition (supervisé) - Construction et entraînement des modèles de prédiction (logistic regression, random forest, XGBoost...) - Évaluation des performances (AUC, recall, precision, confusion matrix) - Sélection du modèle final
Juin	- Génération des scores d'attrition - Développement de dashboards - Création d'interface utilisateur - Rédaction du rapport final et préparation de la soutenance

Tableau 1 : Plan déroulement du projet

1.7 Conclusion

Au cours de ce premier chapitre nous avons présenté l'organisme d'accueil ainsi que le contexte du projet.

Dans le chapitre suivant, nous allons définir les **concepts fondamentaux** du Machine learning et leur application spécifique à l'attrition.



Chapitre II : FONDAMENTAUX DE L'APPRENTISSAGE AUTOMATIQUE

2.1 Introduction

Par nature, l'être humain cherche à tirer des leçons de son passé – échecs comme succès – pour mieux anticiper et maîtriser son avenir. Chaque expérience vécue, avec ses spécificités et son contexte, s'inscrit dans notre mémoire et sert de référence pour guider nos choix face à des situations nouvelles.

Cette capacité d'apprentissage trouve son équivalent scientifique dans le Machine Learning (ou apprentissage automatique). Ce domaine s'appuie sur l'analyse de données (observations, expériences) pour élaborer des modèles capables de classer des éléments ou de prédire des résultats.

Dans ce chapitre, après avoir clarifié quelques notions clés, nous nous focaliserons sur l'apprentissage supervisé et présenterons ses algorithmes les plus utilisés.

2.2 Concepts généraux du ML :

2.2.1 Apprentissage automatique

Le **Machine Learning (ML)**, ou apprentissage automatique, est une branche de l'intelligence artificielle qui permet aux machines d'apprendre à partir de données et d'améliorer leurs performances sans être explicitement programmées pour chaque tâche [1]. Il repose sur des algorithmes capables de repérer des schémas et de faire des prédictions ou des décisions basées sur des données historiques.

Dans ce projet, le ML est appliqué pour :

- Imputation des valeurs manquantes ;
- Prédire le risque d'attrition pour chaque client, sous forme de score.

2.2.2 Types d'apprentissage en Machine Learning

Pour commencer, un algorithme d'apprentissage est représenté par une fonction qui prend en entrée un ensemble d'apprentissage ou d'entraînement D et fournit un classifieur h . L'objectif de l'algorithme A est de généraliser l'information contenue dans D afin de produire une fonction prédictive.

En fonction du type de problème que l'on se pose, on peut mettre en place plusieurs types d'apprentissage.

On distingue deux grands types d'apprentissage :

Apprentissage Supervisé

L'apprentissage supervisé consiste à entraîner un modèle à partir d'un jeu de données étiqueté, c'est-à-dire contenant une **variable cible**. Dans notre cas, il s'agit de prédire si un client va se désengager ou non (attrition = 1 ou 0).

Ce dernier nécessite une base d'apprentissage où chaque instance est préalablement étiquetée avec sa classe respective. Supposons que nous disposons d'un échantillon de données formulés de N variables, nous aurons alors deux types d'informations :

- a. Un vecteur de valeurs $X = (x_1, x_2, \dots, x_N)$ prises par chaque variable ;
- b. Et une valeur de sortie Y appelée valeur supervisée (qui peut être une classe pour le cas d'une classification).

Nous pouvons représenter le problème décrit précédemment par un ensemble d'entrée-sortie (X_i, Y_i) , avec $i = 1 \dots n$, n le nombre d'observations disponibles.

On appelle alors fonction d'apprentissage la fonction notée $l: X \rightarrow Y$ qui associe un résultat supervisé à toute nouvelle entrée qui pourrait lui être présentée.

Apprentissage non supervisé

Ce type d'apprentissage est utilisé lorsque **la variable cible n'est pas connue**. L'objectif est alors de découvrir des structures ou des regroupements cachés dans les données, comme dans le cas de la segmentation client.

Comprenons cela par un exemple. Supposons qu'un chef d'un magasin de location souhaiterait comprendre les préférences de ses clients pour développer son activité. Il pourra regrouper tous ses clients en 10 groupes en fonction de leurs habitudes d'achat et utiliser une stratégie distincte pour les clients de chacun de ces 10 groupes.

2.2.3 Objectifs des méthodes d'apprentissage

Le Machine Learning se décline en plusieurs types de tâches selon la nature du problème à résoudre. Dans le cadre de l'analyse de l'attrition client, les trois principaux types sont : la *classification*, la *régression* et la *segmentation*.

La classification :

La **classification** est une tâche supervisée dans laquelle l'objectif est d'**assigner une observation à une classe prédéfinie**. Elle est particulièrement utilisée lorsque la **variable cible est qualitative (catégorielle)**. C'est la technique utilisée pour prédire si un client risque de se désengager ou non.

Dans le cas de l'attrition, il s'agit d'un problème de classification binaire :

- Classe 0 : client fidèle
- Classe 1 : client à risque (attrition)

Algorithmes classiques :

- Régression logistique : Modèle linéaire pour la classification binaire [2].
- Arbres de décision
- Random Forest : Ensemble d'arbres de décision réduisant le surajustement (*overfitting*) [3].
- XGBoost : Algorithme de *boosting* optimisé pour la performance [4].
- Réseaux de neurones.

Ces méthodes ont démontré leur efficacité pour les problèmes de scoring dans le secteur bancaire [5].

Exemple pratique :

Si un client a un faible niveau d'interactions digitales, un solde moyen décroissant, et peu de produits actifs, le modèle peut prédire qu'il appartient à la classe "attrition".

La régression :

La **régression** est également une tâche supervisée, mais utilisée quand la **variable cible est continue**. L'objectif est de **prédire une valeur numérique** à partir des variables d'entrée. Elle est souvent utilisée pour **prédire un montant, une durée ou une probabilité**.

Dans notre cas, on peut utiliser la régression :

- Pour prédire un **score de probabilité d'attrition** (entre 0 et 1),
- Pour modéliser le **temps avant attrition** (régression temporelle si données disponibles).

Algorithmes typiques :

- Régression linéaire
- Régression logistique (pour une probabilité)
- XGBoost ou Random Forest en mode régressif

Exemple :

Le modèle peut prédire que le risque d'attrition d'un client est de 0,78 → à prioriser dans les actions de rétention.

La segmentation :

La segmentation consiste à **regrouper automatiquement des objets (ici, des clients)** en fonction de similarités dans leurs comportements ou caractéristiques. Contrairement à la classification, il **n'y a pas de variable cible**, c'est une tâche non supervisée.

Cas d'utilisation :

- Créer des groupes de clients homogènes en amont de la modélisation,
- Adapter l'interprétation et des actions à chaque cluster (profiling).

Algorithmes classiques :

- **K-Means** : le plus utilisé pour des clusters sphériques bien définis [6],
- **DBSCAN** : efficace pour détecter des groupes de forme arbitraire et des anomalies [7],
- **GMM** : segmentation basée sur des distributions probabilistes.

Exemple :

Cluster 1 : jeunes clients très digitaux / Cluster 2 : clients anciens à forte épargne / Cluster 3 : clients inactifs.

Résumé comparatif

Type	Supervision	Objectif	Sortie	Exemple dans le projet
Classification	Oui	Prédire une classe	0 ou 1	Client à risque ou non
Régression	Oui	Prédire une valeur réelle	Score (0 à 1)	Probabilité d'attrition
Segmentation	Non	Regrouper les individus	Clusters	Clients similaires selon leurs comportements

Tableau 2 : Type de Machine Learning

2.3 Applications du Machine Learning dans la gestion de l'attrition :

La gestion de l'attrition, qu'elle soit client ou employé, est un défi majeur pour les organisations modernes. Comprendre pourquoi et quand les individus partent est crucial pour la rétention et la croissance. Traditionnellement, cette analyse reposait sur des méthodes réactives, souvent basées sur des enquêtes de sortie ou des analyses statistiques rétrospectives. Cependant, l'avènement du **Machine Learning (ML)** a transformé cette approche, permettant une détection proactive et une intervention ciblée. Cette section explore pourquoi le ML est devenu indispensable dans ce domaine, présente des exemples concrets de son application et détaille les types de données exploitées.

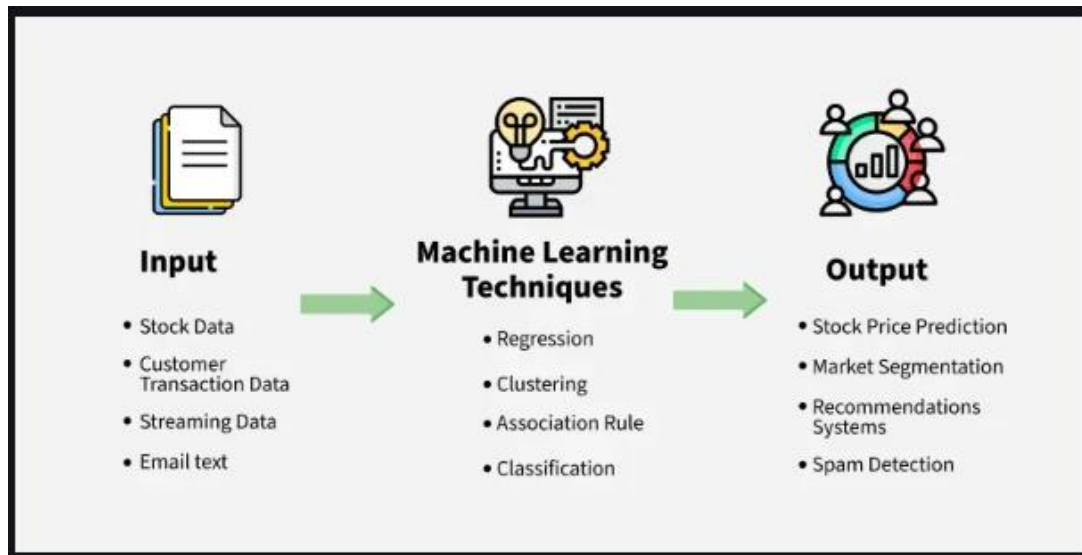


Figure 6 : Machine Learning

2.3.1 Pourquoi le ML pour prédire l'attrition ?

L'intégration du Machine Learning dans la prédiction de l'attrition apporte plusieurs avantages significatifs par rapport aux méthodes conventionnelles, rendant les stratégies de rétention plus efficaces et efficientes.

- Capacité à traiter de vastes volumes de données :** Les organisations collectent aujourd'hui d'énormes quantités de données sur leurs clients ou leurs employés (historiques de transactions, interactions avec le service client, données d'engagement, performance, etc.). Les méthodes traditionnelles peinent à analyser cette complexité. Les algorithmes de ML, à l'inverse, sont conçus pour explorer et tirer des insights de ces jeux de données massifs et multidimensionnels.
- Identification de patterns complexes et non-linéaires :** L'attrition est rarement le résultat d'une cause unique ; elle est souvent influencée par une combinaison complexe de facteurs qui interagissent de manière non-linéaire. Un modèle linéaire simple pourrait passer à côté de ces relations subtiles. Le ML excelle dans la découverte de ces corrélations cachées et de ces dépendances complexes, permettant ainsi une compréhension plus nuancée des déclencheurs de l'attrition.
- Prédiction proactive et intervention précoce :** L'un des atouts majeurs du ML est sa capacité à prédire la probabilité qu'un individu (client ou employé) quitte l'organisation avant que cela ne se produise. Cette capacité prédictive permet aux entreprises de mettre en œuvre des actions de rétention ciblées et proactives. Plutôt que de réagir après le départ, les entreprises peuvent intervenir au bon moment avec des offres personnalisées, un soutien amélioré ou des plans de développement, augmentant ainsi considérablement les chances de rétention.

- **Personnalisation des stratégies de rétention** : Grâce à la segmentation des individus à risque identifiée par le ML, les entreprises peuvent abandonner les approches "taille unique" pour des stratégies de rétention hyper-personnalisées. Cela peut inclure des incitations spécifiques pour les clients, des ajustements de rôle ou des opportunités de développement pour les employés, basées sur les facteurs de risque spécifiques à chaque segment ou individu.

Par exemple, dans le secteur bancaire, un modèle ML peut détecter un désengagement progressif à travers la baisse d'activités sur les comptes, ce que des analyses classiques n'auraient pas anticipé avec la même précision.

2.3.2 Exemples concrets d'utilisation du ML pour l'attrition

Le Machine Learning est appliqué de diverses manières pour anticiper et gérer l'attrition, que ce soit dans le domaine du marketing, du service client ou des ressources humaines.

- **Prédiction de l'attrition client (Customer Churn Prediction)** : C'est l'une des applications les plus courantes. Les modèles de ML analysent les données de transaction (fréquence d'achat, montant, type de produits), les interactions avec le service client (nombre d'appels, résolutions de problèmes), l'utilisation des produits ou services, et même des données démographiques pour identifier les clients à haut risque de désabonnement. Une fois ces clients identifiés, des stratégies proactives comme des offres personnalisées, des appels de suivi ou des améliorations de service peuvent être mises en place. Par exemple, une entreprise de télécommunications pourrait identifier les clients susceptibles de changer d'opérateur en analysant leur historique d'appels, leur consommation de données et leurs plaintes récentes.
- **Détection de l'attrition des employés (Employee Turnover Prediction)** : Dans le contexte des ressources humaines, le ML aide à anticiper quels employés sont susceptibles de quitter l'entreprise. Les modèles peuvent prendre en compte des facteurs tels que le salaire, les promotions, l'évaluation des performances, le nombre de jours de congé, les résultats des enquêtes d'engagement, la distance domicile-travail, et même les données de l'organigramme pour prédire les départs. Cela permet aux RH d'intervenir avec des plans de carrière, des ajustements de rémunération, des programmes de mentorat ou des changements de rôle pour retenir les talents clés.
- **Identification des facteurs de risque d'attrition** : Au-delà de la simple prédiction, le ML aide à comprendre *pourquoi* les individus partent. Les modèles peuvent révéler quels attributs ou combinaisons d'attributs sont les plus fortement corrélés à l'attrition. Par exemple, pour les clients, cela pourrait être une baisse soudaine d'activité ou un certain type de réclamation. Pour les employés, il pourrait s'agir d'un manque de promotion après une certaine période, d'une charge de travail excessive ou d'un faible score d'engagement. Ces insights sont inestimables pour élaborer des stratégies de rétention non seulement réactives mais aussi préventives.

- **Segmentation des populations à risque** : Les modèles de ML peuvent regrouper les individus en différentes catégories en fonction de leur profil d'attrition et des raisons potentielles de leur départ. Par exemple, une entreprise peut identifier un groupe de clients qui partent à cause du prix, un autre à cause d'un mauvais service, et un troisième à cause d'un manque de nouvelles fonctionnalités. Cette segmentation permet de concevoir des interventions de rétention adaptées à chaque profil, maximisant ainsi leur efficacité.

2.3.3 Les différents types de données utilisées pour la prédiction de l'attrition

La richesse des données disponibles est la pierre angulaire des modèles de prédiction de l'attrition basés sur le Machine Learning. La collecte et l'intégration de diverses sources de données permettent de construire une image complète du comportement et des motivations des individus.

Types de données fréquemment utilisées :

- **Données comportementales** : fréquence des connexions, nombre d'opérations, durée d'utilisation des services, etc.
- **Données transactionnelles** : montants moyens, types de produits utilisés, dates de dernière interaction.
- **Données socio-démographiques** : âge, localisation, ancienneté, catégorie professionnelle.
- **Données d'engagement** : taux de réponse aux offres, participation à des programmes de fidélité.
- **Données qualitatives** : résultats d'enquêtes de satisfaction, commentaires clients, etc.

La richesse de ces données permet d'entraîner des modèles prédictifs précis et personnalisés, capables d'anticiper le risque d'attrition avec une grande finesse.

2.4 Conclusion

Ce chapitre a mis en lumière l'importance du **Machine Learning** comme outil prédictif incontournable pour transformer la gestion de l'attrition d'une approche réactive à une stratégie proactive et data-driven. Nous avons exploré les concepts fondamentaux du ML et ses applications concrètes, soulignant comment il permet d'identifier les facteurs de risque et de personnaliser les interventions. Le prochain chapitre sera consacré à **la réalisation** de la solution de prédiction de l'attrition client. Il s'agit de mettre en œuvre les connaissances théoriques pour développer un outil capable d'identifier les clients à risque de départ à partir de données historiques.



Chapitre III : REALISATION DE LA SOLUTION

3.1 Introduction

L'objectif de ce chapitre est de montrer une chaîne de traitement complète : de la collecte et la préparation des données à la mise en ligne d'une application interactive permettant de réaliser des prédictions en temps réel. Cette phase pratique englobe plusieurs étapes clés, notamment le **prétraitement des données**, la **modélisation** avec un algorithme de classification performant (CatBoost), l'**évaluation** des performances du modèle, puis le **déploiement** de la solution à travers une interface utilisateur développée avec Streamlit.

3.2 Sources et nature des données :

Le projet de prédiction de l'attrition client repose sur un ensemble de données internes collectées au sein de l'organisme d'accueil. Ces données sont extraites principalement des systèmes d'information métiers de la banque et concernent différents aspects de la relation client.

3.2.1 Types de données exploitées

Profil & Produit

Ce dataset est un ensemble de données commerciales, lié à la performance de produits et services à chaque fin d'année (2022-2024). Il contient des informations sur les ventes, les revenus et les profits (PNB) ventilés par différentes catégories comme le type de client (CSP), le produit, la région, le pays et le secteur d'activité.

Feature	Description
Année	Représente l'année à laquelle les données se rapportent.
CSP	Catégorie Socio-Professionnelle du client.
ID	Identifiant unique client
Code Produit Pilotage	Utilisée pour l'identification univoque des produits.
Libellé Produit Pilotage	Description textuelle du produit correspondant au "Code Produit Pilotage"
CODE_SSFAMILLE_PRODUIT	Catégorisation plus fine des produits.
LIBELLÉ_SSFAMILLE_PRODUIT	Description de la sous-famille de produit.
Ligne Métier	La ligne d'activité du client (PP, MRE, TPE, etc..)
Mois (#)	Le numéro du mois
Pays Client	Pays d'origine du client
Région	Région géographique
Secteur Activité BAM	Secteur d'activité économique
PNB Vision Commerciale Cumulé	Le Produit Net Bancaire , les revenus générés par l'activité commerciale.
EM C	L' encours moyen cumulés . Cette colonne représente la valeur moyenne des dépôts/crédits du clients en fin d'année.

Tableau 3 : Dataset « Profil & Produit »

Connexion digitale

Ces données contiennent le comportement digital d'un client voire leurs informations tel que leur profil (âge, profession, CSP), la manière dont ils se connectent ou sont acquis (canal de connexion, année de connexion, date d'activation) pour les années (2022-2024)

Feature	Description
ID	Identifiant unique pour chaque client
Année Connexion	L'année au cours de laquelle le client s'est connecté
Date Début Relation	Date à laquelle la relation avec le client a officiellement débuté.
Marché	Le segment de marché auquel le client appartient.
Région	La région géographique du client.
Profession	La profession du client.
Âge client	L' âge du client .
Type Personne	" Physique " pour un individu, " Morale " pour une entreprise ou organisation.
Canal Connexion	Canal par lequel le client s'est connecté : MBK, EBK
CSP	Catégorie Socio-Professionnelle du client.
Date Activation	Date à laquelle le compte ou le service du client a été activé.
Nombre de connexions	Nombre de fois qu'un client s'est connecté

Tableau 4 : Dataset « connexion digital »

Opérations bancaires

Un ensemble de données transactionnelles ou d'opérations distribuées via des agences. Il se concentre sur les volumes et les nombres d'opérations effectué par le client durant 2022 jusqu'à 2024.

Feature	Description
Année	L'année au cours de laquelle les opérations ont eu lieu
Famille opération	La catégorie générale ou le groupe d'opérations .
Type opération	Le type spécifique de l'opération au sein de sa famille
Région	La région géographique où l'opération a été enregistrée
Marché	Le segment de marché du client
Code Agence	Le code identifiant l'agence où l'opération a eu lieu
Volume Opérations en Kdh	Le volume financier total des opérations, exprimé en milliers de dirhams (Kdh)
Nombre Opérations	Le nombre total d'opérations enregistrées
ID	Id du client ordonnateur
Num client BENEf stage	Id du client bénéficiaire

Tableau 5 : Dataset « opération bancaire »

Mouvement financiers / compte courant

Ce dataset se concentre sur les **mouvements de solde de compte du client** en fin d'année de 2022 jusqu'à 2024.

Feature	Description
Année	Représente l' année à laquelle les données (soldes et mouvements) se rapportent.
Mois	Le numéro du mois .
Profession	La profession du client
CSP	La Catégorie Socio-Professionnelle du client
Secteur d'activité	Le secteur d'activité économique auquel le client est lié
ID	Identifiant unique pour chaque client
Solde à M (Dh)	Le solde du compte à la fin de l'année , exprimé en millions dirhams (Dh).
Mouvements Cumulés Crédeurs à M (Dh)	(Entrées d'argent, dépôts) sur le compte du client
Mouvements Cumulés Débiteurs à M (Dh)	(Sorties d'argent, retraits, paiements) sur le compte du client

Tableau 6 : Dataset « Solde & CMC »

Encours Moyen (Base attrition)

Et enfin ce dataset représente un suivi du client axé sur l'engagement financier (encours moyen de ses dépôts) sur une base mensuelle et annuelle (2020-2024)

Feature	Description
Année	Représente l' année à laquelle les données d'encours et de niveau de service se rapportent.
Année entrée en relation du client	L' année au cours de laquelle le client a établi sa relation avec l'entité
ID client	Identifiant unique pour chaque client.
Ligne Métier	La ligne d'activité (TPE, PME...)
Mois (#)	Le numéro du mois
Niveau de Service	Le niveau de service actuel attribué au client.
Niveau de Service Ancien	Le niveau de service précédent attribué au client. (Selon l'ancien système)
Encours Moyen	Représente la moyenne des fonds ou des montants détenus par le client.

Tableau 7 : Dataset « Base Attrition »

3.2.2 Format des données

Avant d'entamer la phase d'entraînement du modèle de classification, il a été nécessaire de consolider les informations disséminées dans les différents datasets. L'approche adoptée a consisté à **pivoter chaque dataset pour s'assurer que chaque ligne représente un client unique**. Cela a permis de regrouper toutes les données pertinentes (financières, comportementales, démographiques) pour un même client.

	Nombre de Ligne Avant pivotage	Nombre de Ligne Après pivotage
Profil & Produit	267 372	25 556
Connexion digitale	43 454	12 593
Opérations bancaires	603 607	22 736
Mouvement financiers / compte courant	64 215	24 900
Encours Moyen	816 799	26 182

Tableau 8 : Tableau comparatif des datasets

Suite à cette transformation, ces datasets individuels ont été **fusionnés en un seul et unique ensemble de données contenant plus de 40 features**. Cette consolidation a créé une vue holistique et complète de chaque client, offrant ainsi un tableau de caractéristiques riche et cohérent, indispensable pour l'entraînement efficace de notre modèle de classification.

Table	Total Rows	Total Colu...
Sheet1	26182	41

Figure 7 : Taille du dataset fusionné

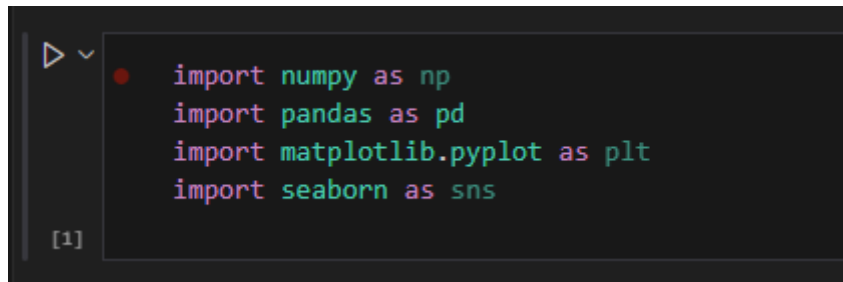
3.3 Préparation des données :

La qualité des données conditionne directement la performance des modèles de Machine Learning. Ainsi, une phase rigoureuse de **préparation** et de **nettoyage** des données a été effectuée en amont de l'entraînement du modèle.

3.3.1 Environnement technique et bibliothèques Python utilisées

Dans le cadre de la préparation des données pour la prédiction de l'attrition, plusieurs bibliothèques Python ont été mobilisées afin de faciliter les différentes étapes du traitement. Ces bibliothèques open source sont largement utilisées dans les projets de data science pour leur efficacité, leur souplesse et leur documentation riche.

- **Pandas** : utilisée pour la manipulation des données tabulaires, la gestion des données manquantes, la création de nouvelles variables, ainsi que le regroupement et l'agrégation d'informations à l'échelle client.
- **NumPy** : utilisée pour certaines opérations numériques et pour optimiser le traitement vectoriel sur de larges volumes de données.
- **Datetime** : utilisée pour la gestion et le traitement des dates, en particulier pour extraire les années, les mois et effectuer des comparaisons temporelles.
- **Scikit-learn** : utilisée pour certaines opérations de prétraitement, notamment l'encodage de variables catégorielles, la séparation en échantillons ou encore l'évaluation des performances du modèle.
- **Matplotlib** et **Seaborn** : utilisées pour la visualisation exploratoire, permettant de mieux comprendre les tendances d'attrition, la distribution des variables et les corrélations.
- **Openpyxl** : utilisée pour générer des fichiers Excel avec les résultats finaux de scoring à destination des utilisateurs métier.
- **Joblib** : utilisée pour la sauvegarde et le chargement du modèle de classification.



```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Figure 8 : Importation des bibliothèques

L'ensemble du traitement a été réalisé dans un environnement Python 3.11, garantissant la compatibilité entre les bibliothèques et la stabilité du projet.

3.3.2 Labellisation de la variable cible (Attrition)

Dans un projet de classification supervisée, la présence d'une **variable cible** est indispensable pour entraîner un modèle. Or, les jeux de données fournis dans le cadre de ce projet ne comportaient **aucune étiquette indiquant explicitement si un client avait quitté la banque (attrition) ou non**.

Pour pallier cette absence, nous avons adopté une **approche heuristique**, c'est-à-dire une méthode empirique reposant sur l'expérience métier, des hypothèses logiques et l'analyse des comportements clients, plutôt qu'une vérité statistiquement observée.

Définition

Une **heuristique** est une méthode de raisonnement ou de prise de décision fondée sur des règles approximatives, souvent utilisées lorsque les données sont incomplètes, floues ou incertaines. En science des données, une **approche heuristique** permet d'inférer ou d'estimer certaines informations à partir de signaux indirects.

Ainsi, la construction de la variable Attrition s'est appuyée sur plusieurs critères observés dans les données :

Règle Métier 1

L'entité Gestion contrôle définissait qu'un client est en attrition en remarquant l'évolution de son encours moyens cumulés de l'année N par rapport à l'encours moyen cumulés de l'année N-1 :

$$\text{Si } ((\text{EMC}_{2023} - \text{EMC}_{2022}) / \text{EMC}_{2022}) * 100 < -90\%$$

Alors

1 : Client considéré en attrition

0 : Client non-attrition

```

#Définir le seuil d'attrition
seuil_attrition = -90

#Créer les colonnes pour Encours Moyen par Année
em_pivot = df.pivot(index='ID client', columns='Année', values='Encours_Moyen_Cumulé').fillna(0)
em_pivot.columns = [f'EM_{int(col)}' for col in em_pivot.columns]

#Créer les colonnes pour nombre de mois avec EM > 0 par an
app_pivot = df.pivot(index='ID client', columns='Année', values='Apparition_EM_Par_Mois').fillna(0)
app_pivot.columns = [f'MoisActifs_{int(col)}' for col in app_pivot.columns]

#Calcul des évolutions EM
years = sorted(df['Année'].unique())
evolution = pd.DataFrame(index=em_pivot.index)
for i in range(1, len(years)):
    prev_year = years[i-1]
    curr_year = years[i]
    evol_col = f'Evolution_EM_{curr_year}vs{prev_year}'
    evolution[evol_col] = (em_pivot[f'EM_{curr_year}'] - em_pivot[f'EM_{prev_year}']) / em_pivot[f'EM_{prev_year}'].replace(0, 1) * 100
    #Forcer les valeurs > 100% à 101%
    evolution[evol_col] = evolution[evol_col].apply(lambda x : 101 if x > 100 else x)
#Ajouter la colonne d'attrition basée sur le seuil
def detect_attrition(row):
    for col in row.index:
        if col.startswith('Evolution_EM_2023vs2022') and row[col] <= seuil_attrition:
            return 1
    return 0
#Finaliser
final_df = pd.concat([em_pivot, app_pivot, evolution], axis=1).reset_index()
#Appliquer attrition
final_df['Attrition'] = final_df.apply(detect_attrition, axis=1)
#Afficher
final_df.head()
  
```

Figure 9 : Script « Attrition règle 1 »

Règle Métier 2

Consiste à rechercher une baisse d'encours moyen pour 3 mois consécutifs durant l'année N, récupérer la valeur minimum de ces 3 mois (EMmin), ensuite récupérer la valeur maximale de l'encours moyen N-1 et la valeur maximale de l'encours moyen N+1. Ainsi nous aurons ces 2 ratios essentielles pour notre règle :

$$\text{Ratio_perte} : (1 - (\text{EMmin}_{2023} / \text{EMmax}_{2022})) * 100$$

$$\text{Ratio_reprise} : (\text{EMmax}_{2024} / \text{EMmax}_{2022}) * 100$$

D'où un client est considéré comme en attrition (valeur de la cible = 1)

- Si $\text{Ratio_perte} \geq 60\%$ and $\text{Ratio_reprise} < 1\%$:

Alors le client est en 'défection'

- Si $\text{Ratio_perte} \geq 80\%$ and $1\% \leq \text{Ratio_reprise} < 20\%$:

Alors le client est en 'attrition'

Et non attrition (valeur de la cible = 0)

- Si $60\% \leq \text{perte} \leq 100\%$ and $20\% \leq \text{reprise} < 40\%$:

Alors le client est en 'pre-attrition'

Sinon

Le client est 'non-attrition'

```
# Ajout : mois à 0 ou manquants en 2022
df_final = df_final.merge(
    df[df['Annee'] == 2022].groupby('Client_ID')['Encours_Moyen'].apply(lambda x: (x == 0).sum()).reset_index(name='mois_2022_a_zero'),
    on='Client_ID',
    how='left'
)

# Ajout : mois actifs en 2023
df_final = df_final.merge(
    df[df['Annee'] == 2023].groupby('Client_ID')['Encours_Moyen'].apply(lambda x: (x > 0).sum()).reset_index(name='mois_2023_actifs'),
    on='Client_ID',
    how='left'
)

def segmenter_client(row):
    perte = row['ratio_perte_2023_vs_2022']
    reprise = row['ratio_reprise_2024_vs_2022']
    mois_2022_a_zero = row['mois_2022_a_zero']
    mois_2023_actifs = row['mois_2023_actifs']

    # Nouveau cas : défection ancienne (hors période d'analyse)
    if mois_2022_a_zero >= 6 and mois_2023_actifs == 0:
        return 'non-attrition'

    # Cas normaux
    if perte >= 60 and reprise < 1:
        return 'défection'
    elif perte >= 80 and 1 <= reprise < 20:
        return 'attrition'
    elif 60 <= perte <= 100 and 20 <= reprise < 40:
        return 'pre-attrition'
    else:
        return 'non-attrition'

df_final['segmentation'] = df_final.apply(segmenter_client, axis=1)

df_final['Attrition_2023'] = df_final['segmentation'].map(lambda x: 1 if x in ['défection', 'attrition'] else 0)

df_final.head()
```

Figure 10 : Script Attrition règle 2

Ces 2 règles métiers nous ont permis la labellisation et de constituer un **jeu de données supervisé**, nécessaire à la phase de modélisation

3.3.3 Test initial sans Feature Engineering

Avant de procéder à une préparation approfondie des données et à la création de nouvelles variables, nous avons réalisé une première série de tests sur le dataset brut (après fusion et ajout du label attrition par **scoring heuristique**). L'objectif était de **mesurer la performance de base** d'un modèle de classification sans enrichissement des données.

Modèle utilisé :

- **Random Forest Classifier**
- Dataset : 41 variables brutes, non transformées
- Label : colonne attrition (issue d'une règle heuristique)

Nous avons testé trois variantes :

1. Random Forest Classique (sans rééchantillonnage)

```

rf_final = RandomForestClassifier(n_estimators=200, random_state=42)
rf_final.fit(X_train, y_train)
y_pred = rf_final.predict(X_test)
y_proba = rf_final.predict_proba(X_test)[:, 1]
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Matrice de confusion:\n", confusion_matrix(y_test, y_pred))

```

	precision	recall	f1-score	support
0	0.87	0.94	0.90	5013
1	0.68	0.45	0.54	1331
accuracy			0.84	6344
macro avg	0.77	0.69	0.72	6344
weighted avg	0.83	0.84	0.83	6344


```

[[4728 285]
 [ 737 594]]

```

Figure 11 : Résultat du 1^{er} test « Random Forest »

Conclusion : Bonne précision globale, mais rappel très faible sur les clients à risque → **beaucoup d'attritions non détectées.**

Rééquilibrage des classes :

L'un des défis majeurs rencontrés dans la prédiction de l'attrition est le déséquilibre des classes : la majorité des clients sont non attritifs, ce qui complique l'apprentissage d'un modèle capable de bien détecter les clients à risque.

Pour atténuer ce déséquilibre, deux approches ont été testées :

SMOTE (Synthetic Minority Over-sampling Technique)

SMOTE est une technique d'oversampling qui génère artificiellement de nouvelles observations pour la classe minoritaire. Au lieu de dupliquer des lignes existantes, SMOTE crée de nouveaux exemples synthétiques en interpolant les caractéristiques des clients à attrition proches les uns des autres.

Avantage :

- Permet d'augmenter la taille de la classe minoritaire sans perte d'information.

Limite :

- Risque d'ajouter du bruit si les données sont très hétérogènes ou non linéaires.

Résultat : (Random Forest avec SMOTE)

```
from imblearn.over_sampling import SMOTE
# Appliquer SMOTE uniquement sur le train
smote = SMOTE(random_state=42)
X_res_sm, y_res_sm = smote.fit_resample(X_train, y_train)

print(f"Avant SMOTE : {y_train.value_counts().to_dict()}")
print(f"Après SMOTE : {y_res_sm.value_counts().to_dict()}")
```

Avant SMOTE : {0: 15436, 1: 5509}
Après SMOTE : {0: 15436, 1: 15436}

```
rf_smote = RandomForestClassifier(random_state=42)
# Entraîner le modèle
rf_smote.fit(X_res_sm, y_res_sm)

# Prédiction sur le test
y_pred_smote = rf_smote.predict(X_test)
y_proba_smote = rf_smote.predict_proba(X_test)[: , 1]

print("=== Résultats avec SMOTE ===")
print(classification_report(y_test, y_pred_smote))
print(confusion_matrix(y_test, y_pred_smote))
```

Figure 12 : Script « Random Forest + SMOTE »

```

=== Résultats avec SMOTE ===
              precision    recall  f1-score   support

      0       0.89         0.87         0.88         5013
      1       0.55         0.60         0.57         1331

 accuracy         0.81         0.81         0.81         6344
 macro avg       0.72         0.73         0.73         6344
 weighted avg    0.82         0.81         0.82         6344

[[4353  660]
 [ 532  799]]
  
```

Figure 13 : Résultat « Random Forest avec SMOTE »

Conclusion : SMOTE améliore légèrement la détection de la classe 1, mais introduit des erreurs sur la classe 0.

Undersampling :

L'undersampling consiste à **réduire le nombre d'observations dans la classe majoritaire** (ici les clients non attrits) afin d'équilibrer le dataset.

Avantage :

- Évite de créer de fausses données.

Limite :

- Risque de **perdre de l'information importante** en supprimant des exemples utiles de la classe majoritaire.

Résultat : (Random Forest avec Undersampling)

```

from imblearn.under_sampling import RandomUnderSampler

rus= RandomUnderSampler(random_state=42)
X_res_rus, y_res_rus = rus.fit_resample(X_train, y_train)

print(f"Avant RUS : {y_train.value_counts().to_dict()}")
print(f"Après RUS : {y_res_rus.value_counts().to_dict()}")
  
```

```

Avant RUS : {0: 15436, 1: 5509}
Après RUS : {0: 5509, 1: 5509}
  
```

```

rf_rus= RandomForestClassifier(random_state=42)
rf_rus.fit(X_res_rus, y_res_rus)
y_pred_rus = rf_rus.predict(X_test)
y_proba_rus = rf_rus.predict_proba(X_test)[: , 1]

print("=== Résultats avec Undersampling ===")
print(classification_report(y_test, y_pred_rus))
print(confusion_matrix(y_test, y_pred_rus))
  
```

Figure 14 : Script « Random Forest + Undersampling »

```

=== Résultats avec Undersampling ===
              precision    recall  f1-score   support

     0           0.93         0.75         0.83         5013
     1           0.46         0.78         0.58         1331

 accuracy                   0.76         6344
 macro avg           0.69         0.77         0.70         6344
 weighted avg        0.83         0.76         0.78         6344

 [[3766 1247]
  [ 287 1044]]
  
```

Figure 15 : Résultat « Random Forest avec Undersampling »

Conclusion : Le rappel de la classe 1 est fortement augmenté, mais **au détriment de la précision globale**. Beaucoup de faux positifs.

Pourquoi ces techniques ?

Ces méthodes sont souvent utilisées **avant l'entraînement d'un modèle**, pour s'assurer que celui-ci ne **biaisera pas les prédictions vers la majorité**. Elles sont particulièrement importantes dans les cas comme le nôtre où **l'objectif est de détecter une minorité stratégique (les clients à risque)**.

Ces tests montrent que le simple rééquilibrage ne suffit pas pour obtenir de bonnes performances. Il faut **comprendre finement les comportements clients**, enrichir les données et tester des algorithmes plus avancés.

Synthèse comparative

Méthode	Recall classe 1	Précision classe 1	Accuracy
Random Forest (de base)	0.45	0.68	0.84
Random Forest + SMOTE	0.60	0.55	0.81
Random Forest + Undersampling	0.78	0.46	0.76

Tableau 9 : Synthèse comparative

Conclusion générale :

Ces premiers essais montrent les **limites du modèle Random Forest sans préparation adéquate des données** :

- Les résultats sur la classe d'intérêt (attrition) sont faibles.
- Les techniques de rééchantillonnage améliorent certains aspects, mais dégradent d'autres.

Ces tests exploratoires justifient la suite du travail : un **travail de feature engineering poussé**, puis le **test d'algorithmes plus robustes** comme **CatBoost**.

3.3.4 Nettoyage et filtrage

- **Suppression des colonnes inutiles** : Les colonnes vides ou ne contenant qu'une seule modalité ont été éliminées, des colonnes tel que : Code Produit Pilotage, CODE_SSFAMILLE_PRODUT, Secteur Activité BAM, Profession, Date Activation, Code Agence.
- **Filtrage des clients** : Les clients n'ayant pas d'activité récente ou dont les informations étaient incomplètes ont été exclus du périmètre d'analyse.
- **Traitement des noms de colonnes** : Les noms de colonnes ont été harmonisés et renommés de manière explicite (ex. : EMC_2023 pour Encours Moyen Cumulé en 2023).

3.3.5 Gestion des valeurs manquantes

Lors de l'analyse exploratoire, la plupart des variables numériques légèrement manquantes ont été **imputées par 0**, notamment lorsque leur absence représentait une absence d'activité. Tel que : les **encours moyens, nombre d'opérations** et **nombre de connexion**.

Nous avons aussi constaté que la variable "**Âge client**" contenait un certain nombre de valeurs manquantes.

Âge client	12388
Année	0
Année entrée en relation du client	0
ID client	0
Ligne Métier	0
Mois (#)	0
Niveau de Service	0
Niveau de Service Ancien	0
Encours Moyen	0

Figure 16 : Affichage des valeurs manquantes

Cette variable étant potentiellement explicative dans le cadre de la prédiction de l'attrition, il était important de procéder à une imputation cohérente.

Problématique initiale

Une première tentative d'imputation par classification (RandomForestClassifier) a été réalisée en prédisant la **catégorie d'âge** à partir des autres variables client (transactionnelles, comportementales et sociodémographiques). L'âge était ensuite généré aléatoirement dans les bornes de la catégorie prédite.

```
# === Préparation X et y ===
X = df_age_known[feature].copy()
y = df_age_known['catégorie_âge'].copy()

# Encodage des variables catégorielles
cat_cols = X.select_dtypes(include='object').columns.tolist()
enc = OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=-1)
X[cat_cols] = enc.fit_transform(X[cat_cols])

# Imputation simple si besoin
num_cols = X.select_dtypes(include=['float64', 'int']).columns
imp = SimpleImputer(strategy='mean')
X[num_cols] = imp.fit_transform(X[num_cols])

# === Modèle ===
clf = RandomForestClassifier(n_estimators=100, random_state=42)
clf.fit(X, y)

# === Préparation des données inconnues ===
X_unknown = df_age_unknown[feature].copy()
X_unknown[cat_cols] = enc.transform(X_unknown[cat_cols])
X_unknown[num_cols] = imp.transform(X_unknown[num_cols])

# === Prédiction catégorie âge ===
predicted_cat = clf.predict(X_unknown)
df.loc[df['Âge client'].isna(), 'catégorie_âge'] = predicted_cat

# === Générer âge aléatoire à partir de la catégorie prédite ===
age_ranges = {
    '<18': (1, 17),
    '18-25': (18, 24),
    '25-30': (25, 29),
    '30-45': (30, 44),
    '45-60': (45, 59),
    '60-80': (60, 79),
}
```

Figure 17: Script « imputation âge (1) »

Cependant, une analyse de la **distribution des âges imputés** a révélé un **biais significatif** :

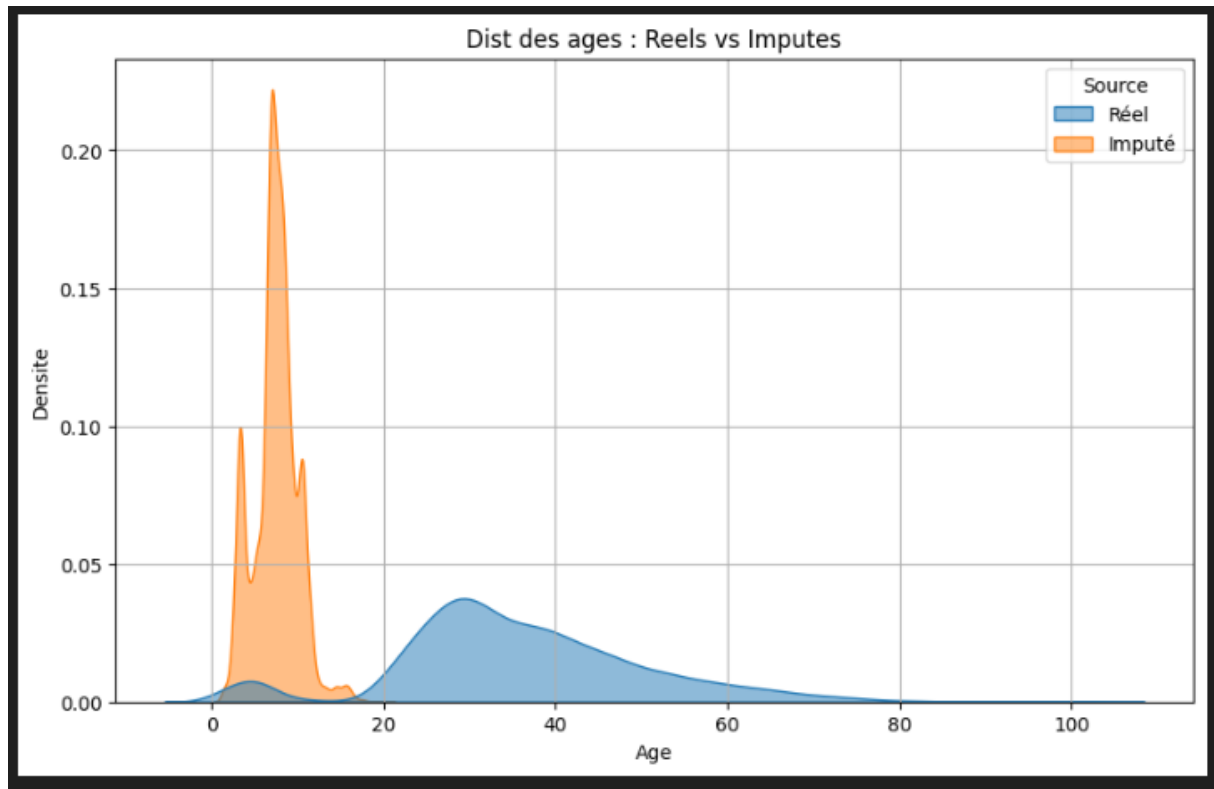


Figure 18 : Graphe « distribution âge (1) »

Plus de 25 % des clients imputés avaient un âge inférieur à 18 ans, contre moins de 5 % dans les données réelles.

Ce déséquilibre montrait que le modèle avait une forte tendance à imputer des clients très jeunes, ce qui était **irréaliste et nuisible à la qualité des données**.

Amélioration de la méthode

Pour corriger ce biais, nous avons :

- Raffiné les **tranches d'âge prédictives** utilisées par le classifieur (<18, 18–25, 25–30, 30–45, 45–60, 60–80, >80)
- Réentraîné le modèle Random Forest uniquement sur les clients avec âge connu
- Généré les âges aléatoires à partir des nouvelles tranches avec des bornes adaptées (ex. : 18–25 → [18, 24])

```

# == Prédiction catégorie âge ==
predicted_cat = clf.predict(X_unknown)
df.loc[df['Âge client'].isna(), 'catégorie_âge'] = predicted_cat

# == Générer âge aléatoire à partir de la catégorie prédite ==
age_ranges = {
    '<18': (1, 17),
    '18-25': (18, 24),
    '25-30': (25, 29),
    '30-45': (30, 44),
    '45-60': (45, 59),
    '60-80': (60, 79),
    '>80': (80, 95)
}

def sample_age_from_category(cat):
    if cat in age_ranges:
        return np.random.randint(age_ranges[cat][0], age_ranges[cat][1] + 1)
    else:
        return np.random.randint(30, 60)

mask = df['Âge client'].isna()
df.loc[mask, 'Âge client'] = df.loc[mask, 'catégorie_âge'].apply(sample_age_from_category)

```

Figure 19 : Script « Imputation âge (2) »

Évaluation de la distribution

Une visualisation des **distributions des âges réels et imputés** a permis de vérifier la qualité de l'imputation.

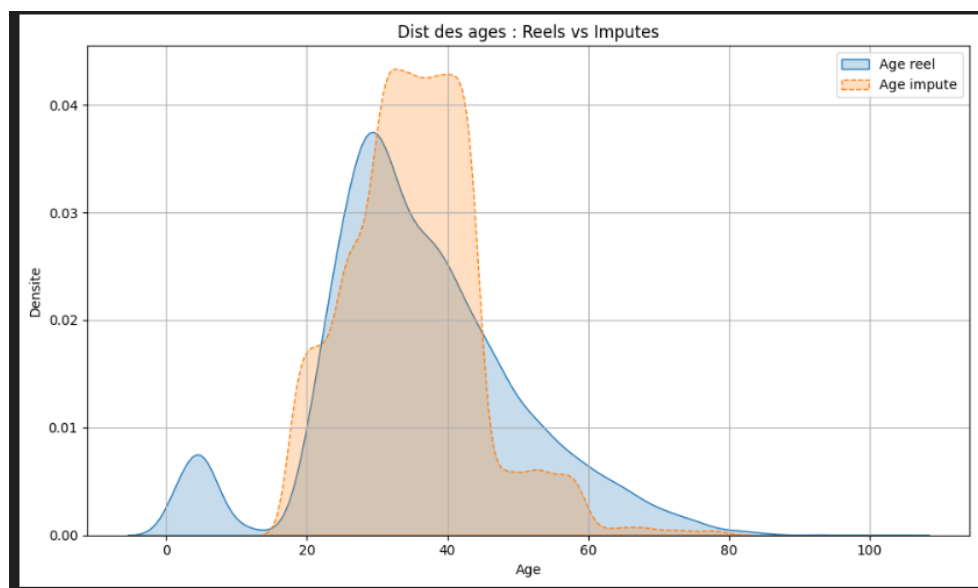


Figure 20 : Graphe « distribution âge (2) »

Le graphique ci-dessus montre que :

- La distribution imputée est désormais **alignée avec les tendances réelles**, notamment entre 25 et 50 ans
- Le **biais d'imputation vers les âges très jeunes a été supprimé**
- Une légère sous-représentation reste observable dans les tranches supérieures à 60 ans, mais elle reste tolérable

3.3.6 Transformation des variables

Les variables catégorielles (Type Personne, catégorie_âge, etc.) ont été **converties en chaînes de caractères** (string) afin d'être correctement traitées par le modèle CatBoost.

```
# 2. Identifier les colonnes catégorielles (non encodées)
cat_features = X.select_dtypes(include=['object', 'category']).columns.tolist()

# Forcer les colonnes catégorielles en string
for col in cat_features:
    if col in df_features.columns:
        df_features[col] = df_features[col].astype(str)
```

Figure 21 : Conversion des colonnes

3.3.7 Construction des variables dérivées (Feature Engineering)

Certaines variables clés ont été générées à partir des données brutes voici quelques exemples :

Features dérivés	Description
EMC_N	Encours Moyen Cumulé du client sur la période d'analyse (en général 2022). Représente le montant moyen détenu par le client.
Evolution_EM_NvsN0	Évolution de l'encours moyen entre l'année d'actuelle (N, par exemple 2022) et l'année précédente (N0, par exemple 2021). Indique la progression ou la baisse des encours.
ZeroStreak_N	Nombre maximum de mois consécutifs où l'encours est resté à zéro durant l'année N. Cela peut signaler une inactivité prolongée.
Delta_EM_N	Différence entre le premier mois et le dernier mois de l'encours moyen dans l'année N. Permet de voir la tendance globale (hausse ou baisse).
MoisActifs_N	Nombre de mois où le client a eu un encours strictement positif dans l'année N.
Volatilite_EM_N	Mesure de la variation de l'encours moyen sur l'année N. Plus la volatilité est élevée, plus l'encours varie d'un mois à l'autre.

PNB_total_N	Produit Net Bancaire total généré par le client durant l'année N. Reflète la rentabilité du client pour la banque.
Nb_produits_différents_N	Nombre de produits bancaires différents détenus par le client (ex. : compte courant, épargne, crédit...).
Nb_ssfamilles_N	Nombre de sous-familles de produits utilisées par le client. C'est un indicateur de diversification de l'usage des services bancaires.
canal_unique	Variable indiquant si le client utilise un seul canal de connexion
Solde_négatif_N	Montant où le client a eu un solde négatif (découvert) dans l'année N.
Nb_Années_Solde_Négatif	Nombre d'années (parmi les années analysées) où le client a eu au moins un solde négatif. Indicateur de comportement risqué sur le long terme.

Tableau 10 : Variables dérivées

3.3.8 Finalisation du dataset

Le dataset final utilisé (de l'attrition 2023) pour la modélisation contient **une ligne par client** avec plus de 70 features.

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 26182 entries, 0 to 26181
Data columns (total 72 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   ID client                                26182 non-null  int64
1   EMC_2020                                26182 non-null  float64
2   EMC_2021                                26182 non-null  float64
3   EMC_2022                                26182 non-null  float64
4   Evolution_EM_2021vs2020                 26182 non-null  float64
5   Evolution_EM_2022vs2021                 26182 non-null  float64
6   ZeroStreak_2020                         26182 non-null  int64
7   ZeroStreak_2021                         26182 non-null  int64
8   ZeroStreak_2022                         26182 non-null  int64
9   Delta_EM_2020                           26182 non-null  float64
10  Delta_EM_2021                           26182 non-null  float64
11  Delta_EM_2022                           26182 non-null  float64
12  MoisActifs_2020                         26182 non-null  int64
13  MoisActifs_2021                         26182 non-null  int64
14  MoisActifs_2022                         26182 non-null  int64
15  Volatilite_EM_2020                      26182 non-null  float64
16  Volatilite_EM_2021                      26182 non-null  float64
17  Volatilite_EM_2022                      26182 non-null  float64
18  Ligne Métier_x                          26182 non-null  object
19  Niveau_Service_Regroupe                 26182 non-null  object
20  Categorie_Volatilite_2020              26182 non-null  object
21  Categorie_Volatilite_2021              26182 non-null  object
  
```

24	total_operations_2021	26182	non-null	int64
25	total_operations_2022	26182	non-null	int64
26	top_famille_opération_2021	26182	non-null	object
27	top_famille_opération_2022	26182	non-null	object
28	top_famille_total_ops_2021	26182	non-null	int64
29	top_famille_total_ops_2022	26182	non-null	int64
30	top_type_opération_2021	26182	non-null	object
31	top_type_opération_2022	26182	non-null	object
32	top_famille_max_volume_2021	26182	non-null	float64
33	top_famille_max_volume_2022	26182	non-null	float64
34	top_famille_agence_2021	26182	non-null	int64
35	top_famille_agence_2022	26182	non-null	int64
36	diversité_opérations_2021	26182	non-null	int64
37	diversité_opérations_2022	26182	non-null	int64
38	operation_vers_autre_banque_2021	26182	non-null	bool
39	operation_vers_autre_banque_2022	26182	non-null	bool
40	ratio_ops_vers_autre_banque_2021	26182	non-null	float64
41	ratio_ops_vers_autre_banque_2022	26182	non-null	float64
42	cat_ratio_ops_vers_autre_banque_2021	26182	non-null	object
43	cat_ratio_ops_vers_autre_banque_2022	26182	non-null	object
44	variation_total_operations_2021_2022	26182	non-null	int64
45	evolution_ratio_ops_autre_banque_2022_2021	26182	non-null	float64
46	A_un_crédit_2022	26182	non-null	int64
47	A_un_dépot_2022	26182	non-null	int64
48	PNB_total_2022	26182	non-null	float64
49	Nb_produits_différents_2022	26182	non-null	int64
50	Nb_ssfamilles_2022	26182	non-null	int64
51	A_un_package_2022	26182	non-null	int64
52	Nb_types_credit_2022	26182	non-null	int64
53	Nb_depots_2022	26182	non-null	int64
54	EMC_credit_2022	26182	non-null	float64
55	CSP_x	26182	non-null	object
56	Pays Client	26182	non-null	object
57	Région	26182	non-null	object
58	Âge client	12593	non-null	float64
59	catégorie_âge	26182	non-null	object
60	canal_unique	26182	non-null	bool
61	nb_connexions_2022	26182	non-null	int64
62	moyenne_connexions_2022	26182	non-null	float64
63	top_canal_2022	26182	non-null	object
64	nb_top_canal_2022	26182	non-null	int64
65	tendance_connexions_2022	26182	non-null	object
66	plus_actif_que_moyenne_2022	26182	non-null	bool
67	type_utilisateur_2022	26182	non-null	object
68	Mouvements_Cumulés_Créditeurs_à_M_(Dh)_2022	26182	non-null	float64
69	Mouvements_Cumulés_Débiteurs_à_M_(Dh)_2022	26182	non-null	float64
70	Solde_à_M_(Dh)_2022	26182	non-null	float64
71	Solde_négatif_2022	26182	non-null	bool

dtypes: bool(5), float64(23), int64(26), object(18)
memory usage: 13.5+ MB

Figure 22 : Dataset Final

3.4 Modélisation :

L'objectif principal de cette phase est de développer un modèle de machine learning capable de prédire avec précision les clients présentant un risque d'attrition. Pour cela, nous avons adopté une démarche expérimentale rigoureuse en testant plusieurs algorithmes de classification supervisée.

3.4.1 Test des modèles

Dans un premier temps, nous avons utilisé **PyCaret**, une bibliothèque open-source en Python qui facilite l'automatisation du cycle de vie des modèles de machine learning.



Figure 23 : Pycaret logo

PyCaret a permis d'effectuer rapidement une comparaison entre différents algorithmes tels que :

- Régression Logistique,
- Random Forest,
- Gradient Boosting,
- Extra Trees,
- LightGBM,
- CatBoost,
- etc.

L'utilisation de PyCaret nous a offert plusieurs avantages :

- Une interface simplifiée pour l'entraînement et l'évaluation des modèles,
- La visualisation automatique des performances à l'aide de métriques telles que **la précision, le rappel, le F1-score et le ROC-AUC**,
- Un **classement automatique des modèles** selon leurs scores.

Suite à cette expérimentation :

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
catboost	CatBoost Classifier	0.8623	0.9266	0.6815	0.7875	0.7304	0.6386	0.6418	8.1620
lightgbm	Light Gradient Boosting Machine	0.8612	0.9263	0.6829	0.7828	0.7292	0.6365	0.6394	36.3530
rf	Random Forest Classifier	0.8575	0.9243	0.6570	0.7882	0.7161	0.6220	0.6270	1.3110
gbc	Gradient Boosting Classifier	0.8546	0.9197	0.6371	0.7917	0.7057	0.6107	0.6172	2.7520
et	Extra Trees Classifier	0.8482	0.9102	0.6408	0.7672	0.6979	0.5977	0.6023	0.8310
ada	Ada Boost Classifier	0.8379	0.9036	0.6293	0.7406	0.6800	0.5725	0.5761	0.6410
lda	Linear Discriminant Analysis	0.8210	0.8581	0.5284	0.7440	0.6175	0.5051	0.5179	0.1210
ridge	Ridge Classifier	0.8208	0.8581	0.4903	0.7726	0.5993	0.4914	0.5128	0.1090
dt	Decision Tree Classifier	0.8100	0.7623	0.6570	0.6526	0.6545	0.5235	0.5237	0.2180
knn	K Neighbors Classifier	0.7345	0.7642	0.4258	0.5190	0.4677	0.2932	0.2958	0.2740
dummy	Dummy Classifier	0.7260	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0900
lr	Logistic Regression	0.6979	0.6910	0.1307	0.3669	0.1853	0.0521	0.0637	1.4890
svm	SVM - Linear Kernel	0.5714	0.5388	0.5715	0.3156	0.3923	0.1014	0.1340	0.1370
qda	Quadratic Discriminant Analysis	0.5285	0.7579	0.8691	0.3617	0.5031	0.1899	0.2680	0.1140
nb	Naive Bayes	0.3717	0.6512	0.7506	0.3202	0.3621	-0.0014	-0.0388	0.1140

Figure 24 : Evaluation des modèles

Le **modèle CatBoostClassifier** s'est avéré le plus performant, notamment sur les critères suivants :

- Gestion native des **variables catégorielles** sans transformation manuelle,
- **Robustesse face aux valeurs manquantes**,
- **Efficacité computationnelle**,
- Bonnes performances sur la **classe minoritaire** (clients en attrition), ce qui est essentiel dans un contexte de déséquilibre des classes.

Le modèle final a ensuite été **reconstruit et entraîné manuellement en dehors de PyCaret**, afin de garder un contrôle total sur les étapes de préparation, d'entraînement et de sauvegarde.

3.4.2 Choix de l'algorithme : Catboost

Le modèle retenu pour ce projet est **CatBoost**, un algorithme de gradient boosting développé par Yandex. Il est particulièrement adapté aux jeux de données contenant des variables catégorielles, ce qui est le cas ici.

Qu'est-ce que CatBoost ?

CatBoost (pour **Categorical Boosting**) est une bibliothèque open-source d'apprentissage automatique basée sur le **gradient boosting** sur arbres de décision.

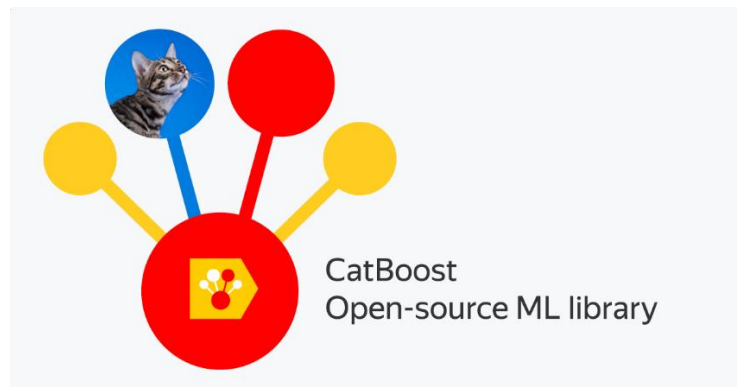


Figure 25 : CatBoost logo

Elle se distingue par :

- **Un traitement natif des variables catégorielles** : pas besoin de recoder manuellement les catégories en one-hot ou en labels.
- **Une performance élevée** : aussi bien en termes de précision que de rapidité d'entraînement.
- **Une bonne gestion des valeurs manquantes.**
- **Une réduction du sur-apprentissage (overfitting)** grâce à des techniques de régularisation intégrées [10].

Fonctionnement

CatBoost repose sur le principe de **boosting**, c'est-à-dire l'entraînement séquentiel de plusieurs arbres de décision, chaque nouvel arbre corrigeant les erreurs des précédents. Contrairement aux autres bibliothèques comme XGBoost ou LightGBM, CatBoost introduit une méthode innovante de traitement des catégories et un schéma d'ordonnancement aléatoire pour améliorer la robustesse du modèle [10].

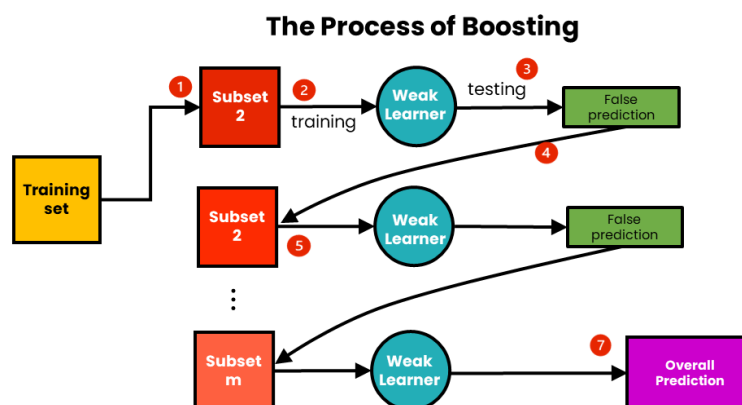


Figure 26 : Principe du Boosting

3.4.3 Réduction du nombre de variables par importance

Avant d'entraîner le modèle final avec CatBoost, une première évaluation de l'importance des variables a été effectuée en utilisant un modèle **Random Forest** sur l'ensemble initial de **72 variables**.

```
df = pd.read_excel(file_path, sheet_name='2023_2')
df.head()
```

[18]:

	ID client	EMC_2020	EMC_2021	EMC_2022	Evolution_EM_2021vs2020	Evolution_EM_2022vs2021	ZeroStreak_2020	ZeroStreak_2021	ZeroStreak_2022	Delta_EM_2020	...	moyenne_connexi
0	1	5.195431e+03	5.190141e+03	6.123909e+03	-0.101822	17.991178	0	0	0	3776.7744	...	
1	2	6.144167e+03	1.122189e+04	2.206286e+03	82.643058	-80.339450	0	0	0	-1510.0953	...	
2	3	1.323133e+05	1.319292e+05	1.315489e+05	-0.290316	-0.288237	0	0	0	-349.1516	...	
3	4	1.220724e+06	1.955356e+06	2.108484e+06	60.180032	7.831186	0	0	0	549170.4463	...	
4	5	8.007897e+04	1.057855e+05	5.328427e+04	32.101520	-49.629914	0	0	0	44674.3808	...	

5 rows x 72 columns

Figure 27 : Extrait du dataset

Tout d'abord nous avons encodé les variables catégorielles en créant une fonction qui permettra cela

```
def preprocess_categoricals(df, target_col=None):
    df = df.copy()

    mappings = {
        "categorie_âge": {
            '<18': 0, '18-25': 1, '25-30': 2, '30-45': 3, '45-60': 4, '60-80': 5, '80+': 6, 'Non Identifié': -1
        },
        "type_utilisateur_2022": {
            'occasionnel': 0, 'digital modéré': 1, 'digital intensif': 2
        },
        "tendance_connexions_2022": {
            'inconnue': 0, 'en hausse': 1
        },
        "cat_ratio_ops_vers_autre_banque_2021": {
            'aucune': 0, 'faible': 1, 'modéré': 2, 'élevé': 3
        },
        "cat_ratio_ops_vers_autre_banque_2022": {
            'aucune': 0, 'faible': 1, 'modéré': 2, 'élevé': 3
        }
    }

    for col, mapping in mappings.items():
        if col in df.columns:
            df[col] = df[col].map(mapping)
```

```
for col, mapping in mappings.items():
    if col in df.columns:
        df[col] = df[col].map(mapping)

onehot_cols = [
    "Categorie_Volatilite_2020",
    "Categorie_Volatilite_2021",
    "Categorie_Volatilite_2022",
    "Ligne_Métier_x",
    "Niveau_Service_Regroupe",
    "top_canal_2022",
    "#Statut",
    "#Type Personne"
]
onehot_cols = [col for col in onehot_cols if col in df.columns]

# OneHotEncoding with drop='first' to avoid multicollinearity
df = pd.get_dummies(df, columns=onehot_cols, drop_first=False)
```

```
df_encoded = preprocess_categoricals(df, target_col='Attrition_2023')
```

Figure 28 : Script « encodage variable catégorielle »

Séparer les X et Y

```

> from sklearn.model_selection import train_test_split
   id_clients = df["ID client"]
   X = df.drop(columns=['ID client', 'Attrition_2023'])
   y = df['Attrition_2023']

# === 4. Train-test split ===
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print('success')
success
  
```

Figure 29 : Séparation X et Y - Feature Importance

Ce modèle a permis d'obtenir un classement des variables selon leur **importance prédictive** (feature importance).

Random Forest

```

1: from sklearn.ensemble import RandomForestClassifier
   from sklearn.metrics import classification_report, confusion_matrix

   # === 5. Feature selection avec RandomForest ===
   rf = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
   rf.fit(X_train, y_train)

1: RandomForestClassifier
   RandomForestClassifier(class_weight='balanced', random_state=42)

[61]: from sklearn.feature_selection import SelectFromModel
      # Sélection des variables importantes
      selector = SelectFromModel(rf, prefit=True)
      X_train_selected = selector.transform(X_train)
      X_test_selected = selector.transform(X_test)
  
```

Figure 30 : Script « Feature Importance »

Résultat :

```
[62]: importances = rf.feature_importances_
feature_names = X_train.columns
feat_imp = sorted(zip(feature_names, importances), key=lambda x: x[1], reverse=True)

# Affichage
sn.barplot(x=[f[1] for f in feat_imp[:20]], y=[f[0] for f in feat_imp[:20]])
plt.title("Top 20 Features")
plt.show()
```

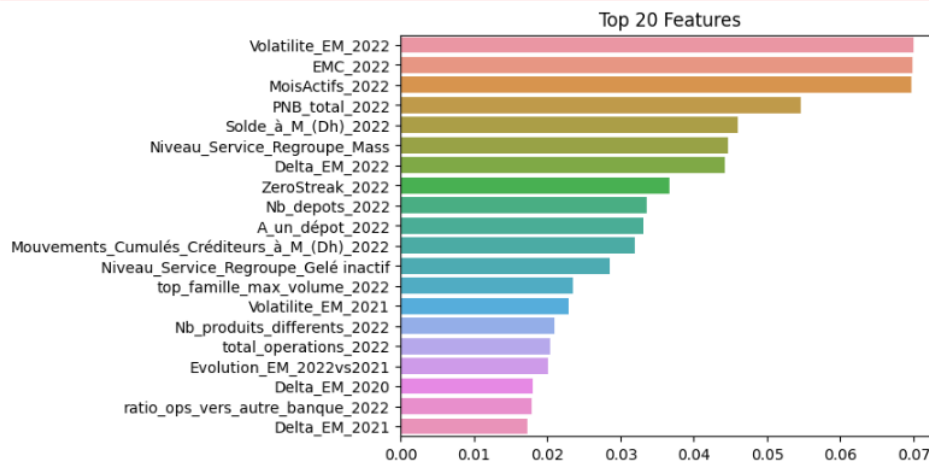


Figure 31 : Feature Importance

Sur la base de ce classement, seules les **28 variables les plus déterminantes** ont été conservées, et le reste a été écarté. Cette réduction a eu pour but :

- De **simplifier le modèle** (moins de complexité, plus de lisibilité),
- De **réduire le risque de sur-apprentissage**,
- Et d'**accélérer le temps de calcul**.

Ainsi, un nouveau modèle (Catboost) a ensuite été entraîné uniquement sur ces 28 variables, et il a produit des **résultats équivalents** en termes de performance (précision, accuracy) à ceux obtenus avec les 72 variables initiales.

```
rf_final = RandomForestClassifier(n_estimators=200, random_state=42)
rf_final.fit(X_train, y_train)
y_pred = rf_final.predict(X_test)
y_proba = rf_final.predict_proba(X_test)[:, 1]
print("Classification Report:\n", classification_report(y_test, y_pred))
print("Matrice de confusion:\n", confusion_matrix(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

     0       0.88       0.94       0.91       3860
     1       0.79       0.63       0.70       1377

 accuracy          0.83
 macro avg          0.79
 weighted avg       0.85

Matrice de confusion:
[[3634  226]
 [ 511  866]]
```

Figure 32 : Résultat du Random Forest avec 72 variables

3.4.4 Entrainement du modèle Catboost

Chargement des données

Cette fois ci nous avons utilisé le dataset réduit à 28 variables pour entrainer notre modèle

[4]:

	ID client	EMC_N	Evolution_EM_NvsN0	ZeroStreak_N0	ZeroStreak_N	Delta_EM_N0	Delta_EM_N	MoisActifs_N0	MoisActifs_N	Volatilite_EM_N0	...	EMC_credit_N
0	1	6.123909e+03	17.991178	0	0	1.191534e+03	-4.065876e+03	12	12	3535.314801	...	0.000000
1	2	2.206286e+03	-80.339450	0	0	5.729041e+03	-6.974068e+03	12	12	13457.091284	...	0.000000
2	3	1.315489e+05	-0.288237	0	0	-3.489388e+02	-3.552164e+02	12	12	114.476288	...	-1685.083327
3	4	2.108484e+06	7.831186	0	0	1.290317e+06	-2.175998e+06	12	12	453797.362144	...	0.000000
4	5	5.328427e+04	-49.629914	0	0	-3.203970e+02	-1.051005e+05	12	12	130.795422	...	0.000000

5 rows × 28 columns

Figure 33 : Extrait dataset (28 variables)

Séparation des données

```

from sklearn.model_selection import train_test_split
id_clients = df["ID client"]
X = df.drop(columns=['ID client', 'Attrition_2023'])
y = df['Attrition_2023']

X_train, X_test, y_train, y_test, id_train, id_test = train_test_split(X, y, id_clients, test_size=0.2, random_state=42, stratify=y)
print(X_train.dtypes)
  
```

Figure 34 : Séparation des données – CatBoost

Nous avons séparé notre jeu de données en deux ensembles : **un ensemble d'entraînement** (environ 80%) et **un ensemble de test** (20%) pour évaluer la généralisation. Le modèle CatBoost a été entraîné en spécifiant explicitement les variables catégorielles identifiées lors de la phase de préparation.

Entraînement

Le modèle CatBoost a été entraîné sur le jeu d'entraînement avec les paramètres suivants :

```

> from catboost import CatBoostClassifier, Pool
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score
import pandas as pd

# Identifier les colonnes catégorielles (non encodées)
cat_features = X.select_dtypes(include=['object', 'category']).columns.tolist()

cat_model = CatBoostClassifier(
    iterations=1000,
    learning_rate=0.05,
    depth=6,
    loss_function='Logloss',
    eval_metric='AUC',
    random_seed=42,
    verbose=100
)

cat_model.fit(X_train, y_train, cat_features=cat_features,
              eval_set=(X_test, y_test), early_stopping_rounds=100)
  
```

Figure 35 : Entraînement du modèle Catboost

Un réglage léger et manuel des hyperparamètres a été effectué, principalement sur le nombre d'itérations, le taux d'apprentissage et la profondeur des arbres, afin d'obtenir un bon compromis entre performance et rapidité.

Détail des hyperparamètres :

Paramètre	Description
iterations=1000	Nombre d'arbres (ou rounds) que le modèle va entraîner. Plus ce nombre est grand, plus le modèle est susceptible de mieux apprendre — mais risque aussi de sur-apprendre si trop élevé.
learning_rate=0.05	Taux d'apprentissage. Plus il est petit, plus l'apprentissage est lent mais précis.
depth=6	Profondeur des arbres. Contrôle la complexité du modèle. Un depth de 6 est un bon compromis entre performance et généralisation.
loss_function='Logloss'	Fonction de perte pour les problèmes de classification binaire.
eval_metric='AUC'	Métrique utilisée pour évaluer le modèle pendant l'entraînement (Area Under the Curve). Approprié en cas de classes déséquilibrées.
random_seed=42	Pour assurer la reproductibilité des résultats.
verbose=100	Affiche les logs toutes les 100 itérations. Pratique pour le suivi.

Tableau 11 : Catboost Hyperparamètres

Ce paramétrage a été choisi de manière arbitraire, après quelques essais avec des valeurs différentes. L'objectif était de garantir un compromis entre **temps d'entraînement**, **capacité de généralisation** et **précision**.

3.5 Évaluation du modèle :

L'évaluation d'un modèle de classification ne peut se limiter à la seule précision globale. Dans un contexte de prédiction d'attrition, il est essentiel de comprendre la capacité du modèle à **détecter correctement les clients à risque (classe 1)** sans générer trop de faux positifs.

3.5.1 Définitions des principales métriques

Voici les principales métriques de classification utilisées pour évaluer le modèle, accompagnées de leur formule :

Matrice de confusion

Tableau croisé entre les valeurs réelles et les valeurs prédites. Elle permet de visualiser les bons et mauvais classements :

	Prédit : Fidèle (0)	Prédit : À risque (1)
Réel : Fidèle (0)	True Negatives (TN)	False Positives (FP)
Réel : Risque (1)	False Negatives (FN)	True Positives (TP)

Tableau 12 : Matrice de Confusion

- **True Positives (TP)** : Le modèle a correctement prédit la classe positive (ex : a prédit « attrition » et le client a bien quitté)
- **True Negatives (TN)** : Le modèle a correctement prédit la classe négative (ex : a prédit « pas attrition » et le client est bien resté)
- **False Positives (FP)** : Le modèle a prédit la classe positive à tort (ex : a prédit « attrition » mais le client est resté)
- **False Negatives (FN)** : Le modèle a prédit la classe négative à tort (ex : a prédit « pas attrition » et le client a quitté)

Accuracy (Exactitude)

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Elle mesure la proportion de bonnes prédictions (clients à risque et non à risque confondus) sur l'ensemble des cas.

Cependant, l'accuracy peut être trompeuse lorsqu'il s'agit d'ensembles de données déséquilibrés, ou une classe l'emporte considérablement sur l'autre.

Points forts de l'accuracy :

- **Interprétation facile** : l'accuracy est facile à comprendre et à interpréter. Exprimée en pourcentage, elle est accessible aux intervenants techniques et non techniques
- **Adapté aux ensembles de données équilibrés** : l'accuracy est une mesure fiable pour les ensembles de données équilibrés, ou chaque classe est représentée de manière à peu près égale. Dans ce cas, elle fournit une image fidèle des performances du modèles [9].

Limite de l'accuracy :

- **Ensemble de données déséquilibrées :** l'accuracy peut être trompeuse dans le cas d'ensembles de données déséquilibrés, ou une classe est nettement supérieur à l'autre. Dans ce cas, un modèle prédisant la classe majoritaire pour toutes les instances peut atteindre une grande accuracy simplement parce qu'il prédit correctement la classe dominante. Cela peut donner une fausse impression d'efficacité du modèle.
- **Inexactitude dans des cas critiques :** dans certains cas, le coût des fautes positives et des fautes négatives peut varier considérablement. L'accuracy traite tous les types d'erreurs de la même manière, ce qui peut ne pas convenir aux situations où les conséquences de chaque type d'erreur diffèrent. Par exemple, dans notre cas, un faux négatif (attrition non détectée) signifie la perte d'un client, tandis qu'un faux positif (client détecté comme attrition à tort) peut entraîner des conséquences moins graves [9].

Quand utiliser l'accuracy

L'accuracy est une mesure précieuse dans les scénarios où l'équilibre des classes n'est pas un problème et où le coût des erreurs de classification est relativement égal pour toutes les classes. Elle est couramment utilisée comme point de départ pour l'évaluation des modèles, mais doit être complétée par d'autres mesures, telles que la précision, le rappel, le score F1 et l'analyse d'une matrice de confusion, pour une compréhension plus complète des performances d'un modèle, notamment dans les applications déséquilibrées ou critiques [9].

Precision (Précision)

$$Precision = TP / (TP + FP)$$

Elle indique parmi les clients prédits comme "à risque", combien le sont réellement. Elle est cruciale pour éviter d'agir inutilement sur de bons clients.

Recall (Rappel ou Sensibilité)

$$Recall = (TP + TN) / (TP + TN + FP + FN)$$

Elle mesure la capacité du modèle à détecter les clients réellement à risque. Un bon rappel est indispensable pour ne pas passer à côté de clients susceptibles de quitter.

F1-score

$$F1 = 2 * (Precision * Recall) / (Precision + Recall)$$

C'est la moyenne harmonique entre précision et rappel. Elle donne une vue globale sur l'efficacité du modèle à détecter les cas positifs.

Importance du score F1 :

- **Gestion du déséquilibre des classes** : Le score F1 est particulièrement utile pour traiter des ensembles de données déséquilibrés, où une classe est nettement supérieure à l'autre. Dans de tels scénarios, un modèle peut atteindre une grande précision en prédisant simplement la classe majoritaire la plupart du temps, mais le score F1 prend en compte les faux positifs et les faux négatifs, offrant ainsi une mesure plus précise des performances globales du modèle.
- **Équilibre entre précision et rappel** : En prenant la moyenne harmonique de la précision et du rappel, le score F1 établit un équilibre entre les deux mesures. Cet équilibre est crucial pour prendre des décisions dans des applications où le coût ou les conséquences des faux positifs et des faux négatifs diffèrent.
- **Métrique unique pour l'évaluation du modèle** : le score F1 condense deux aspects importants des performances d'un modèle en une seule valeur, ce qui le rend pratique pour la sélection du modèle, le réglage des hyperparamètres et la comparaison de différents modèles [9] .

Considération du seuil

Il est important de noter que le score F1 dépend du seuil utilisé pour la classification. Une modification de ce seuil peut avoir un impact sur la précision et le rappel, et donc sur le score F1. Par conséquent, lors de la comparaison des scores F1 entre modèles ou de la prise de décisions concernant le seuil, il est essentiel de prendre en compte le contexte et les priorités spécifiques du problème [9].

Courbe ROC

La courbe ROC (Receiver Operating Characteristic) est une représentation graphique de la capacité d'un modèle de classification à distinguer les classes positives des classes négatives à différents seuils de classification. Elle représente le taux de vrais positifs (TPR), également appelé rappel ou sensibilité, par rapport au taux de faux positifs (FPR), calculé comme la spécificité.

La courbe ROC illustre visuellement comment les performances du modèle changent à mesure que le seuil de classification d'une instance comme positive varie.

Dans la courbe ROC :

- L' **axe des x** représente le **taux de faux positifs (FPR)** , qui mesure la proportion d'instances négatives classées à tort comme positives.
- L' **axe des Y** représente le **taux de vrais positifs (TPR)** , qui mesure la proportion d'instances positives correctement classées comme positives.
- Une courbe ROC typique ressemble à une courbe ascendante, allant du coin inférieur gauche au coin supérieur droit du graphique. **La courbe ROC idéale serait une courbe à angle droit (90 degrés) allant du coin inférieur gauche au coin supérieur gauche** , indiquant une discrimination parfaite entre les instances positives et négatives à tous les seuils [9].

ROC AUC (Receiver Operating Characteristic - Area Under Curve)

Ce score mesure la capacité du modèle à distinguer les deux classes (clients fidèles vs clients à risque), quel que soit le seuil de décision.

Un score de 1 correspond à un modèle parfait, et 0.5 à un modèle aléatoire.

Importance de la courbe ROC et de l'AUC

- **Comparaison de modèles** : la méthode ROC-AUC permet de comparer plusieurs modèles de classification afin de déterminer celui qui est le plus performant. Un modèle avec une AUC plus élevée est généralement plus efficace pour distinguer les classes.
- **Sélection du seuil** : les courbes ROC aident à choisir un seuil de classification adapté aux exigences de l'application. Vous pouvez sélectionner un seuil qui équilibre les TPR et les FPR en fonction du compromis souhaité entre vrais positifs et faux positifs.
- **Ensembles de données déséquilibrés** : les courbes ROC et l'AUC sont particulièrement utiles pour traiter des ensembles de données déséquilibrés, où une classe est nettement supérieure à l'autre. Ces mesures fournissent une évaluation plus complète des performances du modèle, au-delà de la précision.

Limites

Bien que les courbes ROC et l'AUC soient des outils puissants pour l'évaluation des modèles, elles ne permettent pas d'appréhender les conséquences ou les coûts spécifiques associés aux faux positifs et aux faux négatifs. Par conséquent, elles sont souvent utilisées en conjonction avec d'autres indicateurs tels que la précision, le rappel et le score F1 pour mieux comprendre les performances d'un modèle [9].

3.5.2 Résultats du modèle CatBoost

Le modèle final, entraîné avec CatBoostClassifier, donne les performances suivantes :

```
cat_model.fit(X_train, y_train, cat_features=cat_features,  
              eval_set=(X_test, y_test), early_stopping_rounds=100)  
  
# 5. Prédiction  
y_pred = cat_model.predict(X_test)  
y_proba = cat_model.predict_proba(X_test)[:, 1]  
  
# 6. Évaluation  
print("=== Résultats CatBoost ===")  
print(classification_report(y_test, y_pred))  
print("ROC AUC:", roc_auc_score(y_test, y_proba))  
print("Matrice de confusion:\n", confusion_matrix(y_test, y_pred))
```

Figure 36 : Script « Catboost »

Résultat :

```

0:      test: 0.8807381 best: 0.8807381 (0)      total: 86.9ms      remaining: 1m 26s
100:    test: 0.9236069 best: 0.9236069 (100)    total: 2.36s      remaining: 21s
200:    test: 0.9258491 best: 0.9258491 (200)    total: 4.55s      remaining: 18.1s
300:    test: 0.9270645 best: 0.9270775 (299)    total: 6.81s      remaining: 15.8s
400:    test: 0.9277365 best: 0.9277606 (391)    total: 9.12s      remaining: 13.6s
500:    test: 0.9282681 best: 0.9283157 (483)    total: 11.4s      remaining: 11.4s
600:    test: 0.9286571 best: 0.9287504 (584)    total: 13.8s      remaining: 9.16s
Stopped by overfitting detector (100 iterations wait)
  
```

```

bestTest = 0.9287503974
bestIteration = 584
  
```

Shrink model to first 585 iterations.

```

=== Résultats CatBoost ===
              precision    recall  f1-score   support

         0              0.89         0.93         0.91         3616
         1              0.79         0.69         0.73         1364

 accuracy              0.86         0.86         0.86         4980
 macro avg              0.84         0.81         0.82         4980
 weighted avg           0.86         0.86         0.86         4980
  
```

ROC AUC: 0.9287503973866557

Matrice de confusion:

```

[[3365  251]
 [ 426  938]]
  
```

Figure 37 : Résultat « Catboost »

3.5.3 Interprétation des résultats

- Le **taux de bonnes prédictions (accuracy)** atteint **86%**, ce qui est satisfaisant compte tenu du déséquilibre entre classes.
- Le **rappel de 0.69** pour les clients à risque montre que le modèle **identifie près de 70% des clients réellement à risque**.
- La **précision de 0.79** sur la classe "à risque" indique que la majorité des clients prédits à risque le sont effectivement.
- Le **score ROC AUC de 0.93** reflète une excellente capacité de discrimination entre clients fidèles et clients à risque.

La **matrice de confusion** révèle que :

- 938 clients à risque ont été correctement détectés (TP).
- 426 clients à risque n'ont pas été identifiés (FN).
- 251 clients fidèles ont été considérés à tort comme à risque (FP), ce qui reste raisonnable.

Remarque :

Bien que la classe "client à risque" soit minoritaire dans le dataset, le modèle parvient à un **bon équilibre entre détection des cas positifs et limitation des fausses alertes**. Cela permet aux équipes commerciales d'agir avec un certain niveau de confiance, sans trop mobiliser de ressources inutilement.

3.5.4 Evaluation des performances et optimisation

Tout d'abord nous pouvons optimiser les performances de notre modèle, nous avons analysé l'impact du seuil de décision sur les principales métriques d'évaluation : précision, rappel et F1-score. Le modèle générant des probabilités de prédiction, il était essentiel de déterminer le seuil à partir duquel un client est considéré comme en attrition. Pour cela nous avons tout d'abord tracer les courbes d'évolution de chaque métrique afin de trouver leur point d'intersection permettant de trouver le seuil adéquat comme affiche ci-dessous :

```

from sklearn.metrics import precision_recall_curve

# Valeurs réelles
y_true = y_test

# Courbes précision, rappel, f1
precisions, recalls, thresholds = precision_recall_curve(y_true, y_proba)
f1_scores = 2 * (precisions * recalls) / (precisions + recalls + 1e-8)

# Tracé
plt.figure(figsize=(10, 6))
plt.plot(thresholds, precisions[:-1], label='Précision')
plt.plot(thresholds, recalls[:-1], label='Rappel')
plt.plot(thresholds, f1_scores[:-1], label='F1-score', linestyle='--')
plt.xlabel('Seuil de décision (threshold)')
plt.ylabel('Score')
plt.title('Évolution de la précision, du rappel et du f1-score selon le seuil')
plt.legend()
plt.grid()
plt.show()
  
```

Figure 38 : Script « Graphe Seuil de décision »

Résultat :

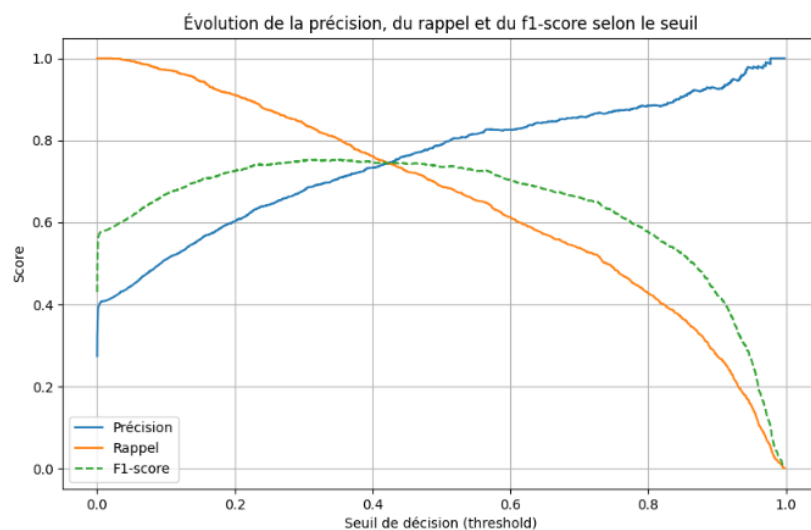


Figure 39 : Graphe « Seuil de décision »

Ainsi cette analyse a permis d'identifier le seuil optimal, offrant le meilleur compromis entre détection des clients à risques (rappel) et limitation des faux positifs (précision). Le F1-score, qui équilibre ces deux aspects, a été utilisé comme indicateur principal pour sélectionner le seuil le plus adapté aux objectifs du projet. En contrepartie cela a réduit la valeur du score ROC-AUC comme affiché ci-dessous :

```

threshold = 0.45
y_pred_thresh = (y_proba >= threshold).astype(int)

# Afficher les résultats
print("=== Évaluation avec seuil =", threshold, "===")
print(classification_report(y_test, y_pred_thresh))
print("ROC AUC:", roc_auc_score(y_test, y_pred_thresh))
print("Matrice de confusion :\n", confusion_matrix(y_test, y_pred_thresh))

```

Figure 40 : Script « Seuil de décision »

Résultat :

```

=== Évaluation avec seuil = 0.45 ===

```

	precision	recall	f1-score	support
0	0.90	0.92	0.91	3616
1	0.76	0.72	0.74	1364
accuracy			0.86	4980
macro avg	0.83	0.82	0.82	4980
weighted avg	0.86	0.86	0.86	4980

```

ROC AUC: 0.8197198667376016
Matrice de confusion :
[[3309  307]
 [ 376 988]]

```

Figure 41 : Résultat du Modèle Catboost avec seuil de décision

Une autre analyse avait été effectuée, dans le but de détecter un éventuel sur-apprentissage (overfitting), nous avons comparé les scores de précision (accuracy) obtenus sur les ensembles d'entraînement et de test. Une différence significative entre les deux performances — notamment une précision nettement plus élevée sur les données d'entraînement — aurait révélé un sur-ajustement du modèle aux données connues.

```
[11]: print("Train accuracy:", cat_model.score(X_train, y_train))
      print("Test accuracy:", cat_model.score(X_test, y_test))
```

Train accuracy: 0.9071285140562249

Test accuracy: 0.8640562248995984

Figure 42 : Test & Train Accuracy

Dans notre cas, le modèle CatBoost a obtenu une précision de **0.90 sur l'ensemble d'entraînement** contre **0.86 sur l'ensemble de test**, ce qui reste raisonnable et indique une **bonne capacité de généralisation** du modèle sur des données nouvelles.

En parallèle, nous avons tracé la courbe d'apprentissage du modèle CatBoost, représentée ci-dessous.

```
# Enregistrement des résultats d'évaluation
evals_result = cat_model.get_evals_result()

# Affichage de la courbe d'apprentissage (AUC ici)
plt.figure(figsize=(10, 6))
plt.plot(evals_result['learn']['Logloss'], label='Train Logloss')
plt.plot(evals_result['validation']['Logloss'], label='Validation Logloss')
plt.plot(evals_result['validation']['AUC'], label='Validation AUC')
plt.xlabel("Iterations")
plt.ylabel("Metric Value")
plt.title("Courbe d'apprentissage CatBoost")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

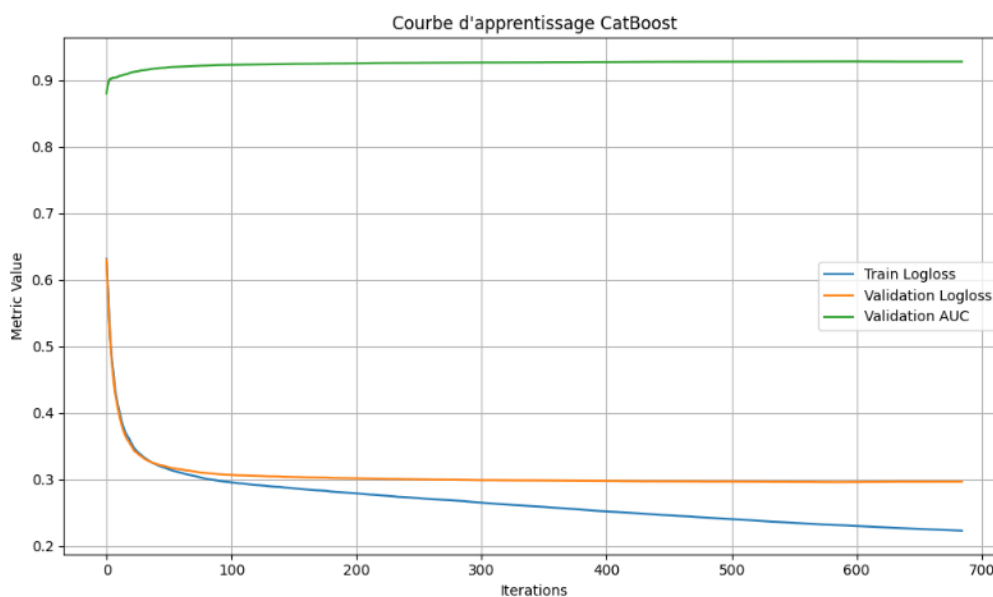


Figure 43 : Script « Courbe d'apprentissage »

Celle-ci montre l'évolution du **logloss sur l'ensemble d'entraînement (ligne bleue)**, du **logloss sur l'ensemble de validation (ligne orange)** ainsi que de l'**AUC sur la validation (ligne verte)** en fonction du nombre d'itérations (arbres).

On observe une **diminution rapide du logloss** dans les premières itérations, suivie d'une phase de stabilisation. Le **logloss de validation se stabilise autour de 0.3**, tandis que le logloss d'entraînement continue légèrement à baisser. Cet écart reste modéré et ne traduit pas un sur-apprentissage marqué. Par ailleurs, **l'AUC de validation dépasse rapidement 0.9** puis reste stable, ce qui indique une **excellente capacité discriminative** du modèle.

Ce graphique a également permis de repérer le moment où le modèle cesse de progresser de façon significative sur la validation, ce qui nous a aidé à appliquer un **early stopping** afin d'éviter un entraînement excessif et d'optimiser le temps de calcul.

La figure ci-dessous présente les courbes ROC (Receiver Operating Characteristic) obtenues pour le modèle **CatBoostClassifier** appliqué à notre problème de classification binaire (attrition vs non-attrition). La courbe ROC permet d'évaluer la performance d'un modèle de classification en représentant le taux de vrais positifs (True Positive Rate) en fonction du taux de faux positifs (False Positive Rate).

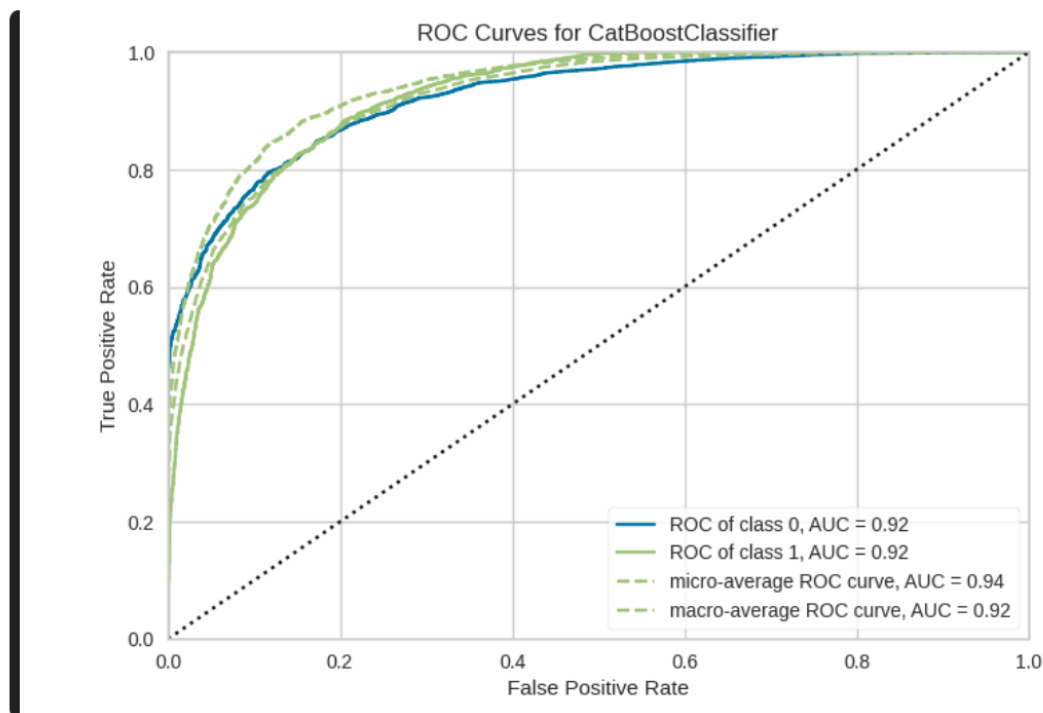


Figure 44 : Graphe « Courbe ROC »

On y distingue :

- **La courbe ROC pour la classe 0 (non-attrition)**, avec une **AUC de 0.92**, ce qui indique une très bonne capacité du modèle à identifier correctement les clients non attrits.
- **La courbe ROC pour la classe 1 (attrition)**, également avec une **AUC de 0.92**, traduisant une performance équilibrée dans la détection des clients en risque d'attrition.
- **La courbe ROC micro-average**, avec une **AUC de 0.94**, qui tient compte de la performance globale sur tous les échantillons, sans distinction de classe. Elle montre que le modèle est performant dans l'ensemble.
- **La courbe ROC macro-average**, avec une **AUC de 0.92**, qui fait une moyenne des AUC des deux classes, confirmant l'équilibre de performance entre les deux classes.

La diagonale en pointillés noirs représente une performance aléatoire ($AUC = 0.5$). Plus la courbe ROC est proche du coin supérieur gauche, meilleure est la performance du modèle. Dans notre cas, toutes les courbes sont nettement au-dessus de cette diagonale, confirmant la **qualité du modèle CatBoost** pour la prédiction de l'attrition.

3.5.5 Comparaison des deux règles de labellisation

Étant donné que le dataset initial ne contenait pas de variable cible (Attrition) directement exploitable, deux **règles métiers heuristiques** ont été définies pour identifier les clients susceptibles d'être considérés comme "à risque". Ces règles ont été construites à partir de l'analyse de l'évolution de l'encours moyen des clients.

Règle 1 : Comparaison annuelle de l'encours (N vs $N-1$)

- Cette règle considère qu'un client est **en attrition** si la **variation de son encours moyen cumulés entre l'année N et l'année $N-1$ est inférieure à -90%**.
- Il s'agit d'un critère simple et strict, basé sur une **chute brutale d'une année à l'autre**.
- **Avantage** : identification claire des pertes importantes.
- **Inconvénient** : ne détecte pas les baisses progressives ou partielles.

Résultats du modèle CatBoost sur la règle 1 :

	precision	recall	f1-score	support
0	0.95	0.92	0.93	4261
1	0.69	0.79	0.74	976
accuracy			0.89	5237
macro avg	0.82	0.85	0.84	5237
weighted avg	0.90	0.89	0.90	5237
[[3917 344]				
[206 770]]				

Figure 45 : Catboost « Règle 1 »

Règle 2 : Analyse dynamique sur 3 années

Cette règle repose sur une analyse plus fine du comportement du client sur trois années consécutives :

1. **Détection de 3 mois consécutifs de baisse d'encours** moyens durant l'année N.
 2. **Calcul de la perte** : comparer le **minimum d'encours moyens pendant la baisse** avec le **maximum de l'encours moyen de l'année N-1**.
 3. **Calcul de la récupération** : comparer le **maximum d'encours moyen de N+1** avec celui de N-1 pour estimer le potentiel de récupération.
- **Avantage** : cette approche tient compte des baisses progressives et des comportements récurrents.
 - **Inconvénient** : plus complexe à implémenter, plus de règles imbriquées, mais mieux alignée avec les dynamiques réelles d'attrition.

Résultats du modèle CatBoost sur la règle 2 :

=== Résultats CatBoost ===

	precision	recall	f1-score	support
0	0.89	0.93	0.91	3616
1	0.79	0.69	0.73	1364
accuracy			0.86	4980
macro avg	0.84	0.81	0.82	4980
weighted avg	0.86	0.86	0.86	4980

ROC AUC: 0.9287503973866557

Matrice de confusion:

```
[[3365 251]
 [ 426 938]]
```

Figure 46 : Catboost « Règle 2 »

Conclusion comparative

- La **règle 1** détecte uniquement les clients ayant subi une **forte perte annuelle**, avec un bon équilibre entre précision et rappel.
- La **règle 2**, bien que plus souple, permet une **meilleure modélisation de comportements progressifs**, ce qui peut s'avérer utile pour **anticiper l'attrition** à un stade plus précoce.

Ainsi, le choix de la règle dépendra de la **tolérance au risque de faux positifs** de la banque et de la **stratégie de rétention** adoptée.

3.6 Déploiement de la solution :

Après l'entraînement et l'évaluation du modèle de prédiction d'attrition, une application interactive a été développée avec **Streamlit**, dans le but de faciliter l'exploitation opérationnelle du modèle par les équipes non techniques.

Streamlit est un **framework open-source en Python** qui permet de créer facilement des interfaces web interactives pour des projets de Data Science et de Machine Learning, sans avoir besoin de compétences en développement web (HTML, CSS ou JavaScript).



Figure 47 : Streamlit Logo

Il permet aux data scientists de transformer rapidement des notebooks ou des scripts Python en applications web, avec des composants tels que des boutons, des formulaires de chargement de fichiers, des graphiques, des tableaux, etc.

Pourquoi l'avoir choisi ?

- Simplicité d'utilisation pour le développeur.
- Intégration native avec des objets Python (pandas, matplotlib, etc.).
- Parfait pour créer une interface légère mais efficace pour :
 - Charger des données,
 - Lancer des prédictions avec un modèle ML,
 - Visualiser les résultats.

Dans le cadre de ce projet, Streamlit a été utilisé pour **déployer le modèle de prédiction d'attrition** et rendre la solution **accessible aux utilisateurs métiers** via une interface web.

3.6.1 Objectifs du déploiement

- Permettre aux utilisateurs de **sélectionner dynamiquement le modèle CatBoost** selon la règle métier choisie (règle 1 ou règle 2).
- Offrir une **expérience simple et fluide** : chargement du fichier → prédiction → analyse → visualisation.
- **Connecter automatiquement les résultats à Power BI** pour des visualisations avancées.

3.6.2 Fonctionnement détaillé de l'application Streamlit

Voici le parcours utilisateur dans l'interface :

Étape 1 – Choix de la règle heuristique

- L'utilisateur choisit entre **deux modèles CatBoost** selon la règle métier utilisée pour labelliser les données :
 - **Règle 1** : comparaison de l'évolution annuelle de l'encours client (N vs N-1).
 - **Règle 2** : détection d'une chute sur 3 mois consécutive, puis analyse de perte et de récupération.

Ce choix conditionne le modèle qui sera appliqué lors de la prédiction.

```

32  === Sélection du modèle ===
33  model_options = {
34      "Catboost v1 (Règle 1)": "D:/Lab_DL/venv_berrada/Attrition/streamlit/catboost_model_score1.pkl",
35      "Catboost v2 (Règle 2)": "D:/Lab_DL/venv_berrada/Attrition/streamlit/catboost_model_score2.pkl"
36  }
37  selected_model_name = st.selectbox("👉 Choisissez le modèle de prédiction :",
38                                     options=list(model_options.keys()))
39
40  selected_model_path = model_options[selected_model_name]
41
42  === Chargement du modèle ===
43
44  Windsurf: Refactor | Explain | Generate Docstring | ✕
45  @st.cache_resource
46  def load_model(model_path):
47      try:
48          model = joblib.load(model_path)
49          st.success(f"✅ Modèle '{selected_model_name}' chargé avec succès !")
50          return model
51      except FileNotFoundError:
52          st.error(f"❌ Erreur : Le fichier modèle est introuvable à l'emplacement : {model_path}. Veuillez vérifier le chemin.")
53          return None
54      except Exception as e:
55          st.error(f"❌ Erreur lors du chargement du modèle '{selected_model_name}' : {e}")
56          return None
57
58  model = load_model(selected_model_path)
  
```

Figure 48 : Script « sélection modèle »

Résultat :

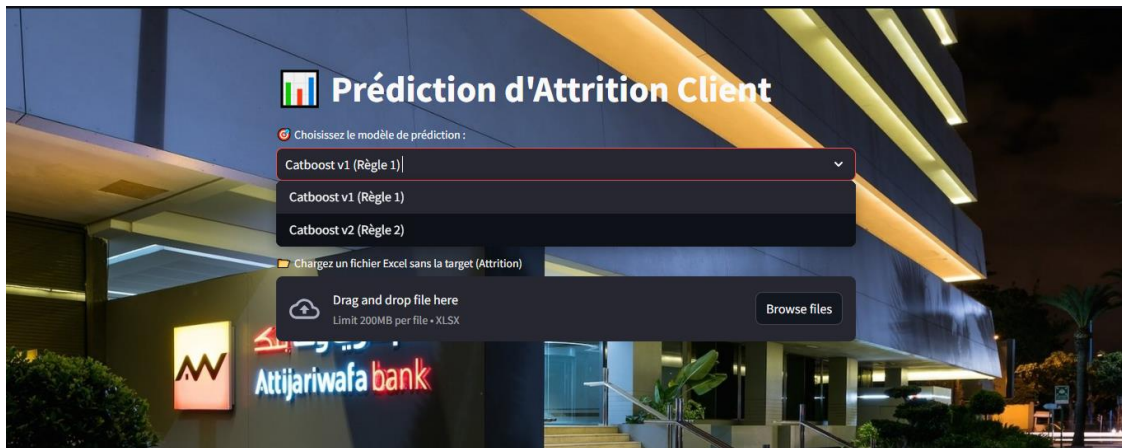


Figure 49 : Sélection du modèle selon la règle.

Étape 2 – Chargement du fichier Excel

- L'utilisateur importe un fichier .xlsx contenant les données clients à scorer.
- Les colonnes inutiles (Unnamed) sont automatiquement supprimées.

```

#=== Upload du fichier utilisateur ===

file = st.file_uploader("📁 Chargez un fichier Excel sans la target (Attrition)", type=["xlsx"])

if file:
    df_raw = pd.read_excel(file)
    df_raw = df_raw.loc[:, ~df_raw.columns.str.contains('^Unnamed')]
    st.write("✅ Fichier chargé avec succès. Dimensions :", df_raw.shape)
  
```

Figure 50 : Script « chargement fichier »

Résultat :

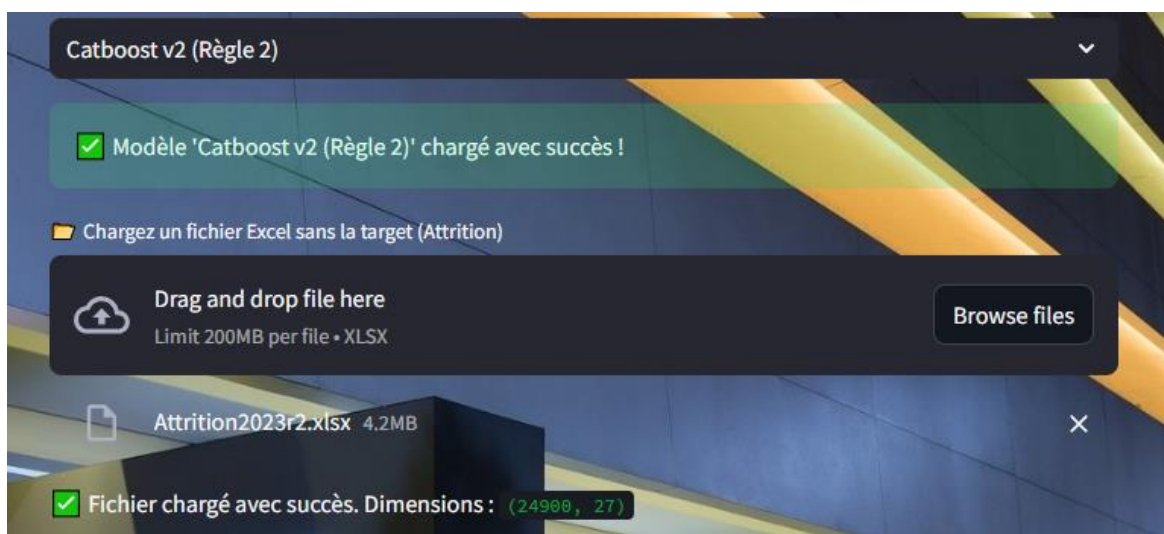


Figure 51 : Chargement du fichier

Étape 3 – Vérification du fichier

Deux boutons apparaissent :

1. Vérifier le fichier :

- Vérifie que le fichier contient toutes les colonnes attendues par le modèle sélectionné.
- Vérifie le type des colonnes catégorielles.
- Signale tout problème (colonnes manquantes ou mal typées).

```

=== Colonnes catégorielles utilisées dans l'entraînement ===

cat_features = [ 'Niveau_Service_Regroupe', 'Type Personne', 'catégorie_âge', 'Ligne Métier_x', 'Statut' ]

# Fonction de validation
Windsurf: Refactor | Explain | Generate Docstring | X
def validate_dataset(df, model, cat_features):
    expected_cols = model.feature_names_
    issues = []

    # Colonnes manquantes
    missing_cols = [col for col in expected_cols if col not in df.columns]
    if missing_cols:
        issues.append(f"🔴 Colonnes manquantes : {missing_cols}")

    # Mauvais types pour colonnes catégorielles
    for col in cat_features:
        if col in df.columns and not pd.api.types.is_string_dtype(df[col]):
            issues.append(f"🔴 La colonne catégorielle '{col}' n'est pas de type string")

    return issues
  
```

```

# Bouton pour vérifier le fichier
if st.button("🔍 Vérifier le fichier"):
    issues = validate_dataset(df_raw, model, cat_features)
    if issues:
        for issue in issues:
            st.warning(issue)
    else:
        st.success("✅ Le fichier est valide pour la prédiction.")
  
```

Figure 52 : Vérification du fichier

Résultat (Colonne Manquante):



Figure 53 : vérification du fichier (Colonne Manquante)

Résultat (Succès) :

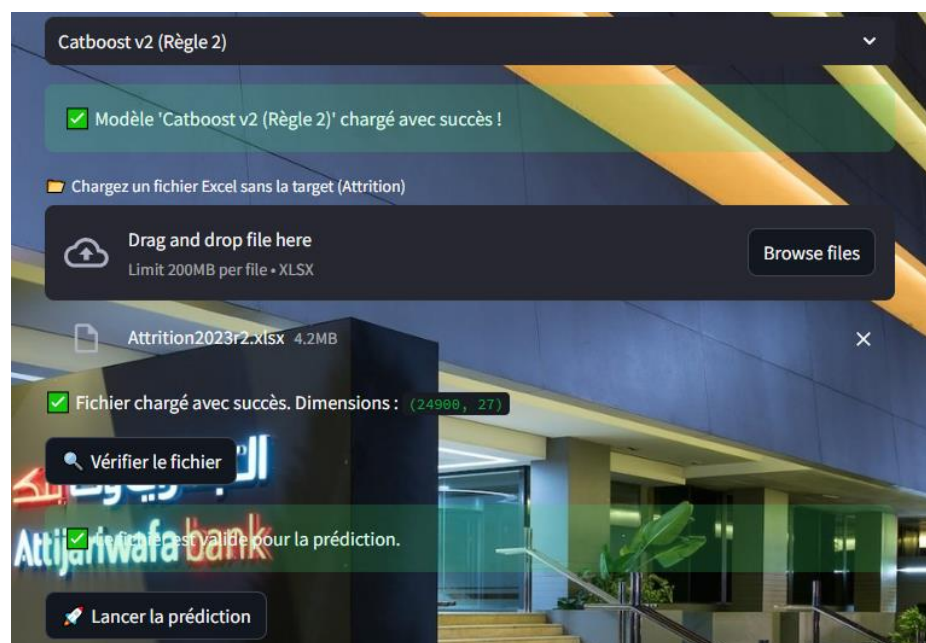


Figure 54 : vérification du fichier (succès)

2. Lancer la prédiction :

- Applique le modèle sélectionné aux données client.
- Deux nouvelles colonnes sont ajoutées :
 - Attrition_Predite : 0 pour client fidèle, 1 pour client à risque.
 - Attrition_Prob : probabilité d'attrition donnée par le modèle.

```

# Bouton pour lancer la prédiction
if st.button("🚀 Lancer la prédiction"):
    try:
        # Préparation
        expected_cols = model.feature_names_
        df = df_raw.copy()

        id_client = df["ID client"] if "ID client" in df.columns else None

        df_model = df[expected_cols].copy() # Réorganise

        for col in cat_features:
            if col in df_model.columns:
                df_model[col] = df_model[col].astype(str)

        df_model = df_model.fillna(0)

        # Prédiction
        y_proba = model.predict_proba(df_model)[: , 1]
        y_pred = model.predict(df_model)

        # Résultat
        df_result = df_raw.copy()
        df_result["Attrition_Prob"] = y_proba
        df_result["Attrition_Predite"] = y_pred
  
```

Figure 55 : Script « Lancer prédiction »

Résultat :

☒ Fichier chargé avec succès. Dimensions : (24900, 28)

☒ Prédiction terminée !

Aperçu des résultats

	M(Dh)_N	Solde_négatif_N	Nb_Années_Solde_Négatif	Age imputé	Attrition_Prob	Attrition_Predite
0	2,532.32	☐	0	☒	9.5388	0
1	671.65	☐	0	☐	55.1935	1
2	129,675	☐	0	☒	92.5414	1
3	342,287.71	☐	0	☐	18.549	0
4	434.91	☐	0	☐	69.014	1

Figure 56 : Résultats de la prédiction avec colonnes ajoutées

Étape 4 – Affichage des résultats

- Un **aperçu du fichier enrichi** est affiché dans l'interface.
- Deux indicateurs clés s'affichent juste en dessous :
 - **Nombre de clients à risque** (valeurs prédites à 1).
 - **Taux d'attrition global** (nombre de clients à risque ÷ total de clients).

```

if "df_result" in st.session_state:
    df_result = st.session_state.df_result
    st.subheader("Aperçu des résultats")
    st.dataframe(df_result.head())

    total_clients = len(df_result)
    at_risk_clients = df_result["Attrition_Predite"].sum()
    attrition_rate = (at_risk_clients / total_clients)*100 if total_clients > 0 else 0

    col_metrics1, col_metrics2 = st.columns(2)

    with col_metrics1 :
        st.metric(label="Total Clients à Risque", value=int(at_risk_clients))

    with col_metrics2 :
        st.metric(label="Taux d'Attrition Global", value=f"{attrition_rate:.2f}%")
  
```

Figure 57 : Script « Aperçu Résultat »

Résultat :



Figure 58 : Taux d'attrition et nombre de clients à risque

Étape 5 – Export et visualisation

Deux boutons terminent le processus :

- **Télécharger les résultats** : télécharge un fichier Excel prêt à être partagé ou archivé.
- **Ouvrir le Dashboard Power BI** : lance automatiquement un fichier .pbix connecté au fichier de résultats pour analyse approfondie.

```

Windsurf: Refactor | Explain | Generate Docstring | X
def convert_df(df):
    output = BytesIO()
    with pd.ExcelWriter(output, engine='openpyxl') as writer:
        df.to_excel(writer, index=False, sheet_name='Résultats')
    output.seek(0)
    # Convertit le DataFrame en bytes pour le téléchargement
    return output

coll, col2 = st.columns([1,1])

with coll:
    st.download_button(
        label="📄 Télécharger les résultats (Excel)",
        data=convert_df(df_result).getvalue(),
        key="download_excel",
        file_name="Attrition_Scoring_Result.xlsx",
        mime="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
    )

with col2:

    pbix_path = "C:/Users/Mohamed Salem/Desktop/template.pbix"

    if st.button("📊 Ouvrir le Dashboard Power BI"):
        try:
            os.startfile(pbix_path)
            st.success("✅ Le fichier Power BI s'est ouvert dans l'application par défaut.")
        except Exception as e:
            st.error(f"Erreur lors de l'ouverture du fichier : {e}")
  
```

Figure 59 : Script « Exportation et Visualisation Power BI »

Résultat :

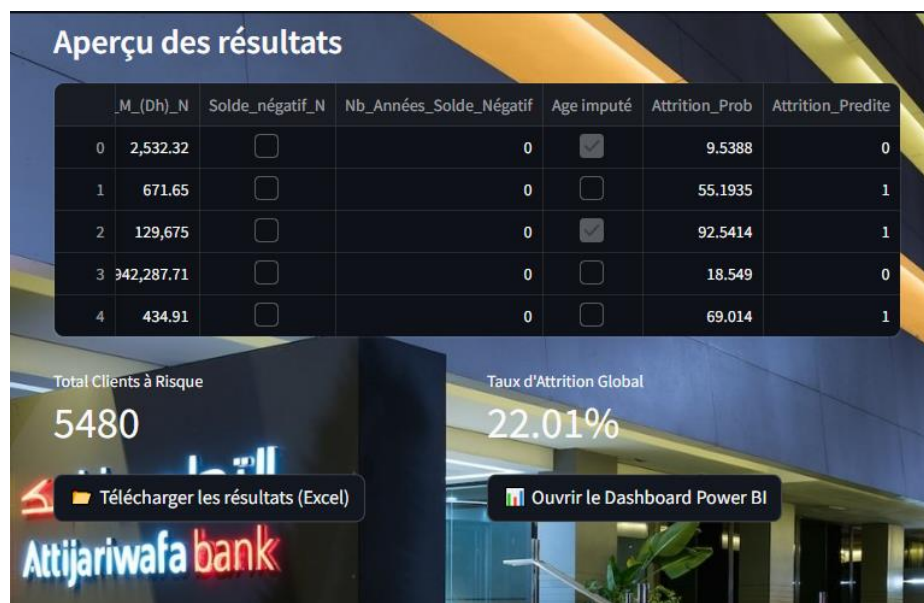


Figure 60 : Boutons de téléchargement et d'ouverture Power BI.

3.6.3 Visualisation des résultats avec Power BI

Deux tableaux de bord ont été développés avec l'outil **Microsoft Power BI** afin de faciliter l'exploitation des résultats par les équipes métiers. Ces dashboards consomment le fichier Excel généré par l'application Streamlit après la prédiction.

Objectifs des dashboards

Les tableaux de bord ont pour but :

- D'offrir une **vue globale** sur la dynamique d'attrition dans le portefeuille client.
- De permettre une **analyse détaillée par client** pour orienter les actions commerciales ciblées.

Data Analysis Expressions :

DAX (Data Analysis Expressions) est le langage de formule utilisé dans Power BI, il se trouve également dans d'autres offres de Microsoft, telles que Power Pivot et SSAS.

DAX est une collection de fonctions, d'opérateurs et de constantes qui peuvent être utilisés dans une formule, ou une expression, pour calculer et retourner une ou plusieurs valeurs. En termes simples, DAX nous aide à créer des informations à partir des données déjà présentes dans le modèle.

Nous avons utilisé les fonctions de ce langage pour générer de nouvelles mesures pour plus d'analyse.

Ci-dessous quelques exemples d'expressions DAX utilisés pour la création des mesures

Mesures	Formules DAX
AttritionRisque	IF(SELECTEDVALUE('Résultats'[Attrition_Predite]) = 1, "En Attrition", "Non Attrition")
NbRisque	CALCULATE(COUNTROWS('Résultats'),'Résultats'[Attrition_Predite] = 1)
PNB_Risque	CALCULATE(AVERAGE('Résultats'[PNB_total_N]),'Résultats'[Attrition_Predite] = 1)

Tableau 13 : Formules DAX

Vue 1 : Dashboard macro/global

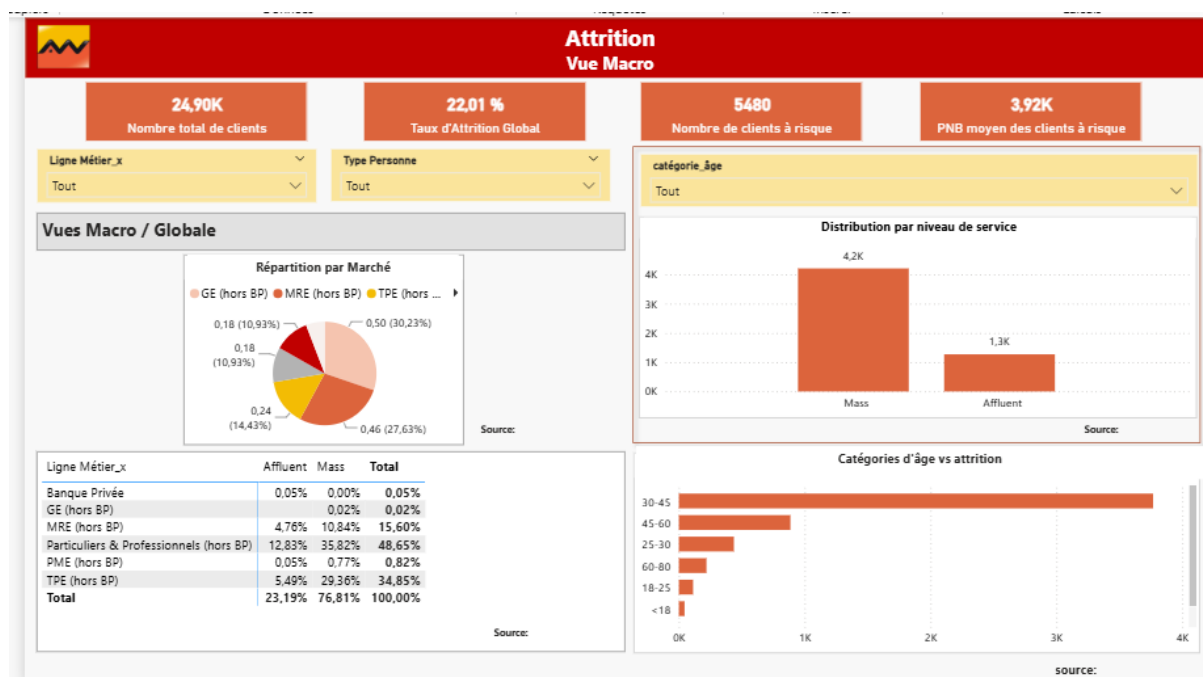


Figure 61 : Vue Macro – Attrition

Ce rapport fournit une vue d'ensemble sur l'état de l'attrition :

- **Taux global d'attrition.**
- **Nombre de clients à risque.**
- **Répartition de l'attrition par segment** : âge, niveau de service, ligne métier, etc.
- **Produit net bancaire moyen des clients à risques.**

Ce dashboard est destiné aux managers ou analystes souhaitant suivre la tendance d'attrition au niveau global.

Vue 2 : Dashboard détaillé par client

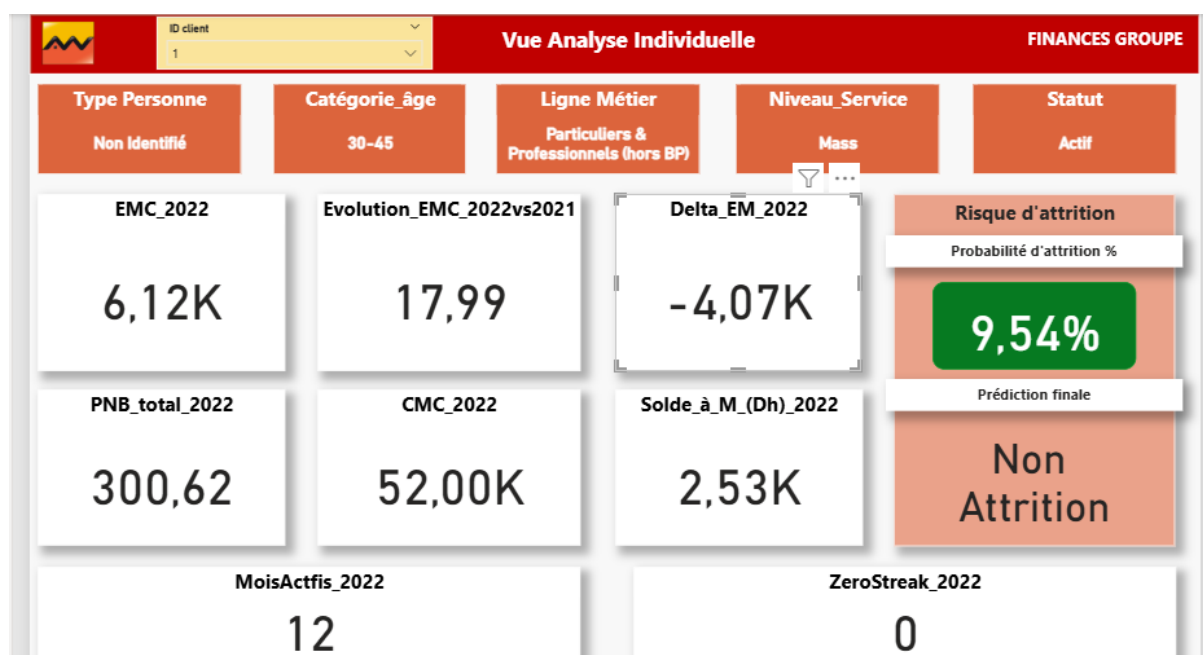


Figure 62 : Vue Individuelle - Attrition

Ce second rapport met l'accent sur **l'analyse individuelle de tous les clients** :

- Fiche signalétique par client.
- Encours et PNB historiques.
- Probabilité prédite d'attrition.
- Visualisation du comportement sur les derniers mois.

Cette vue est conçue pour les conseillers ou chargés de clientèle souhaitant explorer le profil détaillé d'un client avant d'agir.

3.6.4 Avantages de la solution

- **Automatisation** du processus de scoring.
- Interface **intuitive**, accessible à tous les profils métier.
- **Connexion directe à Power BI**, favorisant la prise de décision rapide.
- **Gain de temps opérationnel** significatif pour les équipes marketing ou gestion de portefeuille client.

3.7 Résultats et bénéfices attendus

L'implémentation de la solution de prédiction d'attrition permettra d'atteindre des résultats significatifs, tant sur le plan technique que décisionnel. Les indicateurs produits par le modèle ainsi que les outils de visualisation mis en place offrent une réelle valeur ajoutée pour l'entreprise.

Résultats du modèle

Les performances du modèle CatBoost entraîné sur les données historisées sont satisfaisantes, avec une précision globale de **0.86** et un **AUC de 0.92**, traduisant une bonne capacité à distinguer les clients à risque. Grâce à une approche heuristique de labellisation, deux modèles ont été construits selon deux règles métiers différentes, permettant une flexibilité dans l'analyse du risque :

- **Règle 1** : Comparaison directe entre l'encours moyen de l'année N et celui de N-1.
- **Règle 2** : Détection d'une baisse sur 3 mois consécutive dans l'année N, suivie d'une estimation de la perte et récupération via les pics d'encours de N-1 et N+1.

Ces règles ont permis de générer des jeux de données labellisés réalistes, en l'absence d'un historique explicite d'attrition.

Résultats observés

Après exécution de la prédiction sur un jeu de données réel :

- **Taux global d'attrition détecté** : ~22% (selon la règle 2).
- **Nombre de clients à risque détectés** : **5 480 clients** sur **24 900**.
- **Top segments à risque** : identifiés via le dashboard global (ex. : clients entre 30 - 45 ans).
- **Insight comportemental** : les clients à risque montrent souvent une baisse progressive des encours sur plusieurs mois avant l'attrition.

Bénéfices attendus pour l'entreprise

L'adoption de cette solution de scoring présente de nombreux avantages :

- **Ciblage proactif** : Les équipes marketing peuvent désormais cibler en priorité les clients à risque élevé avec des offres de fidélisation personnalisées.
- **Gain de temps** : Le traitement automatisé via Streamlit permet d'obtenir un scoring prêt à l'usage en quelques secondes.
- **Amélioration du taux de rétention** : Grâce à l'anticipation, il devient possible de déclencher des actions correctives avant la perte du client.
- **Prise de décision data-driven** : Les tableaux de bord Power BI offrent aux décideurs une vision claire et actualisée de l'état de l'attrition.
- **Réutilisabilité** : La solution peut être rejouée sur de nouvelles données sans modifier le pipeline existant.

Retour métier

L'interface conviviale de l'application, associée aux visualisations claires des dashboards, favorise une **adoption rapide par les équipes non techniques**. Elle leur permet d'intégrer la donnée dans leurs décisions quotidiennes sans dépendance à l'équipe IT.

3.8 Limites de la solution proposée

Malgré les résultats satisfaisants obtenus par la solution de prédiction de l'attrition, plusieurs limitations ont été identifiées, tant au niveau des données utilisées que des aspects techniques de la modélisation. Ces limites constituent des points d'amélioration pour une future itération du projet.

1. Qualité et volume des données

- Nous avons travaillé avec des données internes (ex : encours, produits utilisés...), mais il manquait des données plus riches comme satisfaction du client, feedback et d'autres infos personnelles.
- Certaines données étaient manquantes, donc nous avons dû les "remplir" avec des techniques d'imputation, ce qui peut fausser un peu les résultats.

2. Labellisation non supervisée (approche heuristique)

- Le dataset ne disait pas directement quels clients sont partis. Donc nous avons **créé cette info nous-mêmes**, avec deux règles métiers basées sur la baisse des encours.
- Même si ces règles sont logiques, ce ne sont **pas des vérités sûres**. Il y a un risque que certains clients soient mal classés (ex. : un client actif peut être classé comme "à risque", ou inversement).
- Nous n'avons pas validé cette méthode avec un retour réel des équipes commerciales ou relation client.

3. Simplification du modèle

- Nous avons utilisé CatBoost, mais nous ne l'avons pas entièrement optimisé avec des techniques de réglages avancés (tuning approfondi).
- Nous avons réduit le nombre de variables de 72 à 28 en gardant uniquement les plus importantes selon un autre modèle (Random Forest). Mais cette réduction automatique ne garantit pas toujours que nous gardons les variables vraiment utiles pour la banque.

4. Limite de déploiement

- L'application Streamlit est bien pour tester, mais elle n'est pas encore connectée à un système réel ou en ligne.
- Le fichier résultat doit encore être ouvert manuellement dans Power BI. Il n'y a pas d'automatisation complète.
- Nous n'avons pas encore intégré d'alertes automatiques ni de lien avec les outils de gestion client de la banque (comme un CRM).

3.9 Conclusion

La solution développée constitue une base solide pour l'identification des clients à risque d'attrition. Toutefois, la qualité des données, l'approche heuristique de labellisation, et l'absence d'un environnement de production automatisé restent des axes majeurs d'amélioration.

Conclusion Générale

Ce projet de fin d'études s'inscrit dans une problématique réelle et stratégique : **la prédiction de l'attrition client** chez Attijariwafa bank.

Au terme de ce stage, L'objectif principal était de concevoir une solution d'aide à la décision capable d'identifier les clients à risque de départ, en exploitant les techniques du **Machine Learning**.

Au fil des chapitres, nous avons :

- Présenté l'organisme d'accueil (Attijariwafa bank) et l'ensemble du projet (Objectifs, problématiques et contraintes) ;
- Exploré les **concepts fondamentaux** de l'apprentissage automatique et ses apports dans la gestion de la relation client ;
- Étudié et **préparé les données**, en créant des indicateurs métiers pertinents et en labellisant le dataset à l'aide de deux règles heuristiques.
- Mis en œuvre un modèle performant basé sur **CatBoost**, avec une précision globale avoisinant les **86%**, et un **AUC supérieur à 92%**.
- Déployé une interface utilisateur en **Streamlit**, permettant une prédiction simple, intuitive et rapide.
- Construit des dashboards dans **Power BI**, offrant une **vision globale de l'attrition** et une **analyse détaillée par client**, pour soutenir les actions commerciales ciblées.

Sur le plan professionnel, le projet a été d'une très grande utilité. Il nous a permis de **mettre en pratique les compétences acquises et apprises** en data science tout en apportant une **réponse concrète à un enjeu métier**, tout en ouvrant la voie à de futures évolutions technologiques et stratégiques.

Sur le plan humain, ce stage nous a offert la possibilité d'améliorer nos capacités d'adaptation, de communication et de collaboration avec l'environnement professionnel.

Perspectives

Pour aller plus loin, plusieurs axes d'amélioration sont envisageables pour renforcer la solution et l'adapter à un usage opérationnel :

Amélioration du dataset

- Intégrer davantage de données comportementales (ex : interactions avec le service client, feedback, scoring interne).
- Ajouter des données démographiques précises (âge exact, situation familiale, revenus...).

Automatisation du pipeline

- Rendre l'application totalement autonome : récupération automatique des données, prédiction, envoi des résultats et génération de rapports.
- Intégrer des **alertes automatiques** pour signaler les clients à risque aux équipes concernées.

Connexion au système d'information

- Intégrer la solution dans un environnement professionnel connecté au **CRM** ou au **système bancaire**.
- Permettre une exploitation directe des résultats par les départements Marketing ou Service Client.

Validation métier

- Collaborer avec les équipes commerciales pour valider les règles heuristiques utilisées.
- Adapter dynamiquement la définition de l'attrition selon les retours et cas réels

En mettant en œuvre ces améliorations, le projet offrira une valeur ajoutée accrue, contribuant ainsi à une meilleure suivie et anticipation des comportements des clients en attrition et mieux orienter les stratégies de fidélisation client.

Webographie & Bibliographie :

- [0] [Attijariwafa bank, premier Groupe bancaire et financier au Maroc](https://www.attijariwafabank.com/fr) ---
<https://www.attijariwafabank.com/fr>
- [1] Emerj, "What is Machine Learning ?", <https://emerj.com/ai-glossary-terms/what-is-machine-learning/>
- [2] Hosmer, D. W. (2013). Applied Logistic Regression. Wiley.
- [3] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- [4] Chen, T. (2016). XGBoost: A Scalable Tree Boosting System. arXiv:1603.02754
- [5] Géron, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 2e édition, 2019.
- [6] MacQueen, J. (1967). *Some Methods for Classification and Analysis of Multivariate Observations*.
- [7] Ester, M. (1996). *A Density-Based Algorithm for Discovering Clusters*. KDD.
- [8] Hastie, T., Tibshirani, R., & Friedman, J. *The Elements of Statistical Learning*, Springer, 2009.
- [9] Classification Metrics -- <https://www.geeksforgeeks.org/machine-learning/sklearn-classification-metrics/>
- [10] CatBoost , <https://www.geeksforgeeks.org/machine-learning/catboost-ml/>
- [11] Müller, A. & Guido, S. *Introduction to Machine Learning with Python*, O'Reilly, 2016.
- [12] scikit-learn documentation: Clustering Algorithms — <https://scikit-learn.org/stable/modules/clustering.html>
- [13] Scoring Heuristique,
<https://datafranca.org/wiki/Heuristique#:~:text=En%20informatique%20et%20en%20intelligence,un%20probl%C3%A8me%20d'optimisation%20difficile>.
- [14] Predicting customer churn using machine learning: A case study in the software industry -- [Predicting customer churn using machine learning: A case study in the software industry | Journal of Marketing Analytics](#)
- [15] Memoire de Fin d'étude Master, Prédiabète : Un Système de Détection et prédiction de diabète -- [SAHLI_SOUHA_F5_1657103614.pdf](#)
- [16] Customer churn prediction in telecom sector using machine learning techniques, [Customer churn prediction in telecom sector using machine learning techniques - ScienceDirect](#)
- [17] Building a Churn Prediction Model (Theoretical Guide) -- [Building a Churn Prediction Model \(Theoretical Guide\) | by Amit Yadav | Biased-Algorithms | Medium](#)
- [18] What is Churn prediction? -- [Churn Prediction Made Simple & Top 9 ML Techniques](#)
- [19] Pycaret -- [pycaret/tutorials/Tutorial - Binary Classification.ipynb at master · pycaret/pycaret · GitHub](#)
- [20] Customer Churn Prediction in the Banking Sector Using Machine Learning-Based Classification Models
https://www.researchgate.net/publication/368911804_Customer_Churn_Prediction_in_the_Banking_Sector_Using_Machine_Learning-Based_Classification_Models
- [21] CUSTOMER CHURN PREDICTION -- International Advanced Research Journal in Science, Engineering and Technology, Vol. 8, Issue 6, June 2021